

Data API

Formunauts App

Version 2.9
March 4, 2026

Formunauts GmbH
Theobaldgasse 20/1/7
1060 Wien, Austria
info@formunauts.at

Content

[Content](#)

[Version history and change log](#)

[About the API](#)

[Authentication](#)

[Pagination](#)

[Filtering](#)

[Rate Limiting](#)

[Alternative Addresses](#)

[Additional Fields](#)

[Sponsorships](#)

[Downloading PDFs](#)

[API endpoints](#)

[List donations](#)

[Confirm retrieval of donations](#)

[List unfinished donations](#)

[Confirm retrieval of unfinished donations](#)

[List donation feedback](#)

[Confirm retrieval of Donation Feedback](#)

[List petitions](#)

[Confirm retrieval of petitions](#)

[List time tracking entries](#)

[Confirm retrieval of time tracking entries](#)

[List fundraisers](#)

[Add a new fundraiser without fundraiser image](#)

[Add a new fundraiser with password](#)

[Add a new fundraiser and upload a fundraiser image](#)

[Set/update the image of a fundraiser](#)

[Set a fundraiser active/inactive](#)

[Edit Fundraiser](#)

[List teams](#)

[Add a new team](#)

[Edit a team](#)

[Delete a team](#)

[Add a fundraiser shift](#)

[Edit a fundraiser shift](#)

[Delete a fundraiser shift](#)

[Listing available customers](#)
[Listing available campaigns](#)
[Listing available petition campaigns](#)
[Listing available work shifts](#)
[Listing available locations](#)
[Add a new location](#)
[Add a location with an image](#)
[Edit a location](#)
[Set/update the image of a location](#)
[Delete a location](#)
[List job applications](#)
[Confirm retrieval of job applications](#)
[List Recruiting Campaigns](#)

[Webhooks](#)

[Outgoing Webhooks](#)
[Incoming Webhooks](#)

[Appendix A: Status Codes](#)

[Appendix B: Status Reasons](#)

[Misc](#)
[Approved](#)
[Held](#)
[Cancelled or Failed](#)

Version history and change log

Version 2.9 (March 4, 2026)

- Added `sign_up_coordinates` to Donation and Unfinished Donation endpoints

Version 2.8 (July 1, 2025)

- Added `is_formunauts_one` to Campaign List endpoint
- Added new fields `raisenow_payment_uuid`, `raisenow_subscription_uuid`, `amount_covered` to Donation list endpoint for epms payments, marked fields `raisenow_epp_transaction_id`, `raisenow_subscription_id`, `raisenow_subscription_canceled` for deprecation
- Removed legacy `feedback_rating_choice` field from Donation endpoint
- Removed section about Legacy Donation Feedback Webhook

Version 2.7 (November 5, 2024)

- Added `team_id`, `fundraiser_id`, `direct_debit_amount` fields to Donation list endpoint
- Added `team_id`, `fundraiser_id` fields to Petition list endpoint
- Added `team_id`, `fundraiser_id`, `customer_id` fields to Job Application list endpoint
- Added `status` and `status__exclude` filters to Donation list endpoint
- renamed `welcome_call_qq_fundraiser_handed_anything` to `welcome_call_qq_opt_ins_confirmed` field in Donation list endpoint

Version 2.6 (May 8, 2024)

- fixed parameter names of createtime filter introduced in Version 2.5

Version 2.5 (February 25, 2024)

- Added Filtering section describing the new filters based on createtime
- Clarified TLS version in About the API section
- Added information about throttling in the Authentication section

Version 2.4 (October 17, 2023)

- Added `?is_active` filter for Fundraiser list endpoint
- Added `tax_identification_number` to Donation
- Removed `gift_recipients` from Donation

Version 2.3 (November 30, 2022)

- Add rate limiting to Donation, Team and Fundraiser endpoints.
- Added `stripe_customer_id`

- Added `raisenow_subscription_id`

Version 2.2 (May 10, 2022)

- Removed deprecated fields on Donation list endpoint: `on_hold`, `on_hold_comment`, `bank_account_was_validated`
- Added new status fields to Donation list endpoint
- Removed `on_hold`, `on_hold_comment`, added status field on Petition list endpoint
- Added `quality_check_passed` field to Donation endpoint if quality check is enabled on the campaign
- Added `status_comment`, `status_reason`, `external_status_reason`, `original_payment_method` fields to Donation list endpoint if the campaign has dataflow enabled
- Added optional `welcome_call` fields to Donation list endpoint if the campaign has dataflow enabled
- Replace single `campaign` field with multiple `campaigns` field on Team endpoint if the customer has multi campaign setup enabled.
- Added `'carer'` as option to `donor_occupation` in Donation and Petition list endpoints

Version 2.1 (September 14, 2021)

- Extended possible `donor_salutation` values to include "Dr" for List donations and List petitions endpoints.
- Extended available `team_type` options in Teams endpoints for "private_site" and "event"
- Extended available `campaign_type` options in Donation, Petition and TimeTracking list endpoints.
- Added 2 optional fields for campaigns with active self-checkout configuration in List donations endpoint: `"sco_subscription_active"`, `"sco_journey_completed"`

Version 2.0 (June 15, 2021)

- breaking change in authentication: use POST body instead of url parameter to submit `grant_type=client_credentials`.
- We moved all endpoints from `https://donutapp.io/api/v1/` to `https://app.formunauts.com/api/v2/`
- Added Configurable Webhooks for Donations and UnfinishedDonations.
- Added incoming Webhooks for updating payment status of Donations.
- Added date filter to List team endpoint

Version 1.35 (March 15, 2021)

- Added documentation for downloading PDFs

- Added endpoint for listing DonationFeedback
- Fixed Listing after retrieval of `error` through `confirm_retrieval` endpoint, as described in documentation
- Allow deleting of fundraiser shift with existing Location

Version 1.34 (November 23, 2020)

- Added `team_tags` to List Donations

Version 1.33 (August 11, 2020)

- Added optional additional fields to Locations

Version 1.32 (March 30, 2020)

- Added `donor_department` and `donor_district` to List Donations, Unfinished Donations and Petitions

Version 1.31 (February 5, 2020)

- Added `donor_address_addition`, `donor_address_addition_2`, `donor_province` to List Donations, Unfinished Donations and Petitions
- Added documentation for undocumented Fields on Fundraiser: `username`, `language`, `staffcloud_refid`, `staffcloud_import_date`, `image_thumbnail_url`
- Removed deprecated `address` Field of Fundraiser List
- Renamed previously undocumented field `sextant_vendor_id` to `fundraiser_external_id` and added to docs.
- Added `fundraiser_external_id` to “List donations” and “List unfinished donations”.
- Changed behavior of Delete a Fundraiser Shift request
- Added missing possible values to `welcome_email_status`
- Added documentation for previously undocumented fields on Petition: `welcome_email_status`, `on_hold`, `on_hold_comment`

Version 1.30 (March 30, 2019)

- Added `bank_account_bank_name` to List Donations

Version 1.29 (December 13, 2018)

- Fixed description of `direct_debit_interval` field

Version 1.28 (December 11, 2018)

- Changed JSON structure of teams regarding coaches
- Added `raisenow_epp_transaction_id`

Version 1.27 (September 20, 2018)

- Added section for editing Fundraiser

Version 1.26 (September 10, 2018)

- Added `donation_amount_annual_in_original_currency` and `donation_original_currency` to List Donations

Version 1.25 (August 20, 2018)

- Added `payment_history` to List Donations

Version 1.24 (August 6, 2018)

- Added `total_pages` parameter to Pagination
- Added `Accept` Header to example requests

Version 1.23 (August 3, 2018)

- Updated `Authentication` chapter

Version 1.22 (August 1, 2018)

- Added occupation choices

Version 1.21 (July 26, 2018)

- Added `donations_count` to `team_fundraiser_work_shifts` (only if location tracking is enabled)

Version 1.20 (July 23, 2018)

- Added `uploadtime` to `/donations`, `/petitions` and `/jobapplications` endpoints

Version 1.19 (July 2, 2018)

- Added optional location information and `has_logged_in` fields to `team_fundraiser_work_shifts` in List teams

Version 1.18 (June 19, 2018)

- Added `contact_data_conf` field

Version 1.17 (June 14, 2018)

- Added `donor_age_in_years` and `applicant_age_in_years` field

Version 1.16 (May 28, 2018)

- Renamed `adress` field of Fundraiser to `street`
- Added `consent_given` field to Fundraiser

Version 1.15 (May 3, 2018)

- Added `image_thumbnail_url` to `/teams/` endpoint
- Added `image_thumbnail_url` to `/campaigns/` endpoint.

- Added `image_thumbnail_url` to `/petition_campaigns/` endpoint.
- Added `points_label`, `currency`, `locale` to `/customers/` endpoint.

Version 1.14 (March 26, 2018)

- Added `tags`, `locations`, `petition_campaigns`, `prolong_workshift_setup`, `goal_value_unit` and `goal_value` to `/teams/` endpoint.
- Added endpoint for listing `petition_campaigns`

Version 1.13 (February 15, 2018)

- Added `gift_recipients` to `/donations/` endpoint.
- Added `name` field to `/teams/` endpoint

Version 1.12 (January 15, 2018)

- Added section about flexible fields.
- Added `applicant_salutation` to `/jobapplications/` endpoint.
- Added `sponsorships`.

Version 1.11 (November 20, 2017)

- Added the `donor_profile.update` webhook to the Quality Widget webhooks.

Version 1.10 (August 16, 2017)

- **Important Change:** Locations are now not related to the campaign but only to the customer. You have to give a customer ID. (See location section for more information)
- The `/fundraisers/` endpoint has now also the `PUT` method available. So `GET`, `POST`, `PUT` and `PATCH` are available.
- The `/locations/` endpoint has now also the `PATCH` method available. So `GET`, `POST`, `PUT`, `PATCH` and `DELETE` are available.
- The list of campaigns is now always sorted by the creation date (oldest first)
- The list of breaks in the time entries are now always sorted by `shift_start`
- List of time entries: The coach is now returned (the coach used to be always `null`)
- List of time entries: Time entries with an retrieval error will not be listed (like with donations and petitions)
- Confirming of time entries with an error: This can be done now. Before there was always a server error returned when trying to confirm an time entry with an error.
- Added Endpoints for listing and confirming Unfinished Donations
- Added fields to Donation endpoint: `bank_account_was_validated`, `campaign_type2`, `payment_method`, `change_note_private`, `change_note_public`, `raisenow_status`, `raisenow_subscription_canceled`
- Added fields to Petition Endpoint: `campaign_type2`, `change_note_private`, `change_note_public`

Version 1.9.1 (July 21, 2017)

- Added parameter to pagination to control page size

Version 1.9 (July 20, 2017)

- Added Job Applications list & confirm
- Added Recruiting Campaigns Endpoint
- Added description for further values of welcome_email_status

Version 1.8 (July 4, 2017)

- Changed location endpoint and deprecated campaign in location

Version 1.7 (June 20, 2017)

- Added chapter about pagination
- Added chapter about alternative address delivery if address validation is enabled

Version 1.6.2 (June 6, 2017)

- Added donor_company_name field to Donation

Version 1.6.1 (April 10, 2017)

- Added on_hold field to Donation
- Added Campaign Prefix to person_id

Version 1.6 (March 28, 2017)

- Added webhooks
- Some minor spelling fixes

Version 1.5.1 (December 1, 2016)

- Added feedback_rating, feedback_rating_choice, feedback_rating_custom_answer fields to Donation

Version 1.5 (August 5, 2016)

- Added possibility to create fundraisers with passwords through API
- Added CRUD Operations for Locations
- Added locations Parameter to Teams
- Added Update Operations for Teams & Fundraiser Shifts

Version 1.4 (June 10, 2016)

- Added the following fields to donations data: contract_start_date, location, welcome_email_status, comments_qa, confirm_direct_debit, confirm_bank_account, confirm_satisfied, confirm_information_check, confirm_fundraiser_incentive_information
- Added the following fields to petitions data: welcome_email_status

Version 1.3 (May 10, 2016)

- Added documentation for managing fundraisers.
- Added documentation for managing teams.
- Added documentation for listing campaigns and work shifts.

Version 1.2 (February 9, 2016)

- Added documentation to handle petitions.

Version 1.1 (July 7, 2015)

- Time tracking entries can now have multiple breaks. So the properties `start_time_break` and `end_time_break` were replaced with a `breaks` array containing all the breaks. See [Listing time tracking entries](#)
- For convenience the properties `work_hours`, `break_hours` and `num_breaks` were added to the list of time tracking entries. See [Listing time tracking entries](#)

Version 1 (June 3, 2015)

- Initial Version

About the API

With the Donut data API it is possible to export records of different kinds and manage teams and workshifts of your fundraisers.

The Donut Data API is a JSON API that can be accessed via HTTPS. The client is required to support TLS version 1.2

The base URL of the API is <https://app.formunauts.com/api/v2/>. At this location you will also find a browsable version of the API that you can access with your formunauts api service account.

Authentication

You will receive a `client_id` and a `client_secret` with your Data API account.

With these credentials you can request an access token. You have to make a HTTP POST request, providing the `client_id` and `client_secret` via Basic Authentication:

Here a `curl`¹ example you can use as a demo:

```
curl -i -X POST "https://app.formunauts.com/o/token/" -u
"9Di4rUBubHpMQpkm4eANs9Wg9ycNUUXWjfxC3Nv1:4cg0UjtJ2wVNcXqOhXCbk7vj0jd
MlsKr7GtbApPvDvKHmVYjShl842BZsjSg3xCKksUQWxKtJe8oagcEodXltpqBUFSFv5jI
7Ek1jxXfYoZVPFkXx4oYlRXWYAk123zp" -d "grant_type=client_credentials"
```

IMPORTANT NOTICE REGARDING VERSION 2.0

There is a breaking change to the api authentication: Because of updating the underlying oauthlib package, starting from June 15th 2021 the `/o/token` endpoint will not accept any GET parameters anymore. Instead the required `"grant_type=client_credentials"` parameter must be submitted in the POST body.

This request will return a JSON object with your `access_token` and some additional information:

```
{
  "access_token": "ZoGvK0RBdJnqxoItlqT9KnygmYek3v",
  "token_type": "Bearer",
  "expires_in": 36000,
  "scope": "read write"
}
```

The `expires_in` property shows how long this `access_token` is valid in seconds. Every access token is valid for 10 hours, after this time the token expires and you have to request a new access token.

¹ <http://curl.haxx.se/>

You can now make authenticated requests to the Data API by adding the `access_token` in the “Authorization” header of your requests. A simple example:

```
curl -i -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" https://app.formunauts.com/api/v2/donations/
```

IMPORTANT NOTICE

Since Version 2.5 of the API released on 27th February 2024 we throttle the requests to `/o/token` and limit them to 6 requests per client and minute. When this limit is reached we return a HTTP 429 (Too many requests) until the lock is removed.

Pagination

Data is paginated into sets of 10 records by default. You can use the parameter `?page_size=[n]` to control the size of the delivered pages. The param can be set to any positive integer between 1 and 100.

Outside the “results” set pagination information is delivered:

```
{
  "count": 37,
  "total_pages": 4,
  "next": https://app.formunauts.com/api/v2/donations/?page=2,
  "previous": null,
  "results": [...],
}
```

count	Integer	Grand total of delivered records
total_pages	Integer	Total count of pages (depending on chosen page size)
next	String	Link to the next page of donations, or null.
previous	String	Link to the previous page of donations, or null.
results	List	Array of JSON objects representing the donations.

IMPORTANT NOTICE

Records are generally listed ascending by createtime. If you query the list and there are some concurrent actions taken, records might appear on different pages.

Example 1:

While querying the list of donations, a new donation is uploaded and cleared for retrieval through the Data API. Consequence: The new donation is inserted at the head of the list, all other donations are subsequently rowed back. A donation that might has appeared on the 10th place on the first page, will now appear on the 1st place on the 2nd page.

Countermeasure:

- Always check the uid of the donation. This is always unique, so if you already read the donation with the given uid from the API, you should not process it a second time

Example 2:

While querying the list of donations, some donations are confirmed as retrieved.

Consequence: This will lead to the remaining donations to move up in the list. Eg. if the retrieval of 10 donations is confirmed, there will be 1 less total page.

Countermeasures:

- Keep the processes of retrieving and confirming the donations in sync
- Consider either calling only the first page and confirming the records, and afterwards again query the first page (the donations are consequently moved up), or first query all pages and then confirm all donations (see the note in the '[Confirm retrieval of donations](#)' section of this document on confirming many donations at once)

Filtering

In Version 2.5 we added filtering based on `createtime` to various endpoints.

Example request:

```
curl -i -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqx0ItlqT9KnygmYek3v" https://app.formunauts.com/api/v2/donations/?date\_from=2024-02-01&date\_to=2024-02-27
```

This will limit the output to donations with a `createtime` between 2024-02-01 00:00 and 2024-02-27 23:59

The endpoints which have `createtime` based filtering enabled are:

- `/api/v2/donations/`
- `/api/v2/donations/unfinished/`
- `/api/v2/jobapplications/`
- `/api/v2/petitions/`

In Version 2.7 we also added filtering based on `status` to the Donation list endpoint:

- the `?status` filter as a comma separated list of stati the returned Donations should have
- the `?status__exclude` filter as a comma separated list of stati the returned Donations should *not* have

See [Appendix A: Status Codes](#) for a list of possible values

Example request:

```
curl -i -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqx0ItlqT9KnygmYek3v" https://app.formunauts.com/api/v2/donations/?status=call, live
```

Rate Limiting

Some endpoints are rate limited. Note that this is counted per user across all the limited endpoints.

The following list & detail endpoints are subject to this rate limiting:

- Donations
- Fundraisers
- Teams

Paginated endpoints have a limit of 60 requests/minute while detailed ones, e.g. get Donation PDF, have a limit of 600 requests/minute.

When you exceed the limit you will receive a response code of 429 and can expect a body like this one.

Example response:

```
{
  "detail": "Request was throttled. Expected available in 60 seconds."
}
```

Alternative Addresses

When the address validation module is activated (for customers with the professional package), the API gets enriched with extra fields representing the results of the address validation.

This applies to both **list donations** and **list petitions** endpoints.

Example response:

```
{
  "donor_zip_code": "123123",
  "donor_street": "Bloomsbury",
  "donor_house_number": "12",
  "donor_city": "London",
  "donor_country": "UK",
  ...
  "primary_address": "original",
  "address_quality": 66,
  "alternative_street": "Bloomsbury Square",
  "alternative_house_number": "12",
}
```



```

    "alternative_zip_code": "WC1A 2LP",
    "alternative_city": "London",
    "alternative_country": "GB"
}

```

Explanation of the additional fields:

primary_address	String	Guides which address is delivered in the donor_ fields and which are delivered in the alternative_ address block. Can either be "original" or "validated". In the above example the original address is used because the validated address differs too much from the address the user entered on the tablet. The address needs review. NOTE: In case the validated address can not be set automatically (either the difference between entered and validated addresses is too big or the address could not be found at all) the status of the record will be set to "held".
address_quality	Integer	The similarity between the original and validated address, in percent.
alternative_street	String	The alternative street name
alternative_house_number	String	The alternative house number.
alternative_zip_code	String	The alternative zip code.
alternative_city	String	The alternative city.
alternative_country	String	The alternative country.

Additional Fields

Introduced in Version 1.12 we are now able to add a flexible set of Fields to the Forms and to the according API endpoints ([List donations](#), [List petitions](#), [List job applications](#)).

The key can be configured freely in the campaign setup and will be added to the resulting object. If multiple fields use the same identifier the described values will be appended and divided by a semicolon. The following table gives you an overview of the available fields and their output in the API.

Form Input Type	API Output
Checkbox	Configured identifier string or <code>null</code> if not selected
Radio button group	String value of chosen option or <code>null</code> if no option is selected. <code>true/false</code> is delivered as boolean values for backwards compatibility reasons with 'confirm_' options (see List donations , List petitions)
Textfield	The value that was entered as string. There is also the option to prefix the field with an identifier which will be prepended to the string value and delimited by a colon (":")
Range slider	Selected value as Integer.

JSON Example

```
{
  ...
  "special1": "checkbox1:yes;checkbox2:no;textfield2:asdf",
  "confirm_direct_debit": true,
  "radiogroup2": null,
  "textfield1": "Lorem ipsum ...",
  "range slider1": 5
}
```

Sponsorships

Sponsorships are added to the result set in a similar fashion as the additional fields. The field name and the value (if chosen) can be configured in the campaign setup. Also a default key/value pair if no sponsorship is chosen can be set.

Downloading PDFs

Some API endpoints return a PDF URL through which you can download the according PDF of the record. The response of this endpoint is `binary data with application/pdf` MIME Type.

URL: retrieved through list endpoint

Method: GET

Example request:

```
curl -i -H "Accept: application/pdf" -H "Authorization: Bearer  
ZoGvK0RBdJnqxoItlqT9KnygmYek3v"  
https://app.formunauts.com/api/v2/donations/pdf/?uid=2028212078
```

API endpoints

List donations

Returns a list of the donations in the system that are ready to be fetched via the API.

URL: /donations/

Method: GET

Example request:

```
curl -i -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" https://app.formunauts.com/api/v2/donations/
```

Example response:

```
{
  "count": 37,
  "total_pages": 4,
  "next": "https://app.formunauts.com/api/v2/donations/?page=2",
  "previous": null,
  "results": [{
    "uid": 2028212078,
    "person_id": "GP12313123",
    "organisation_id": "123121",
    "agency_id": "123131",
    "customer_id": 6,
    "campaign_id": 21,
    "campaign_type": 1,
    "campaign_type2": "city_campaign",
    "donor_first_name": "Thomas",
    "donor_last_name": "Coram",
    "donor_company_name": "Dr. Coram Inc.",
    "donor_academic_title": "Dr.",
    "donor_salutation": 2,
    "donor_sex": 1,
    "donor_date_of_birth": "1668-01-01",
    "donor_age_in_years": 350,
    "donor_occupation": 4,
```

```

"donor_email": "thomas.coram@foundling.org",
"donor_phone": "01222222222",
"donor_mobile": "06762222222",
"donor_zip_code": "123123",
"donor_street": "Bloomsbury",
"donor_house_number": "12",
"donor_city": "London",
"donor_department": "",
"donor_province": "Greater London",
"donor_district": "",
"donor_address_addition": "Apt. 2",
"donor_address_addition_2": "",
"donor_country": "UK",
"contact_by_phone": 0,
"contact_by_email": 0,
"fundraiser_code": "d-00789",
"fundraiser_name": "Donut, James",
"fundraiser_id": 47,
"fundraiser_external_id": "ext-186019081899",
"donation_amount_annual": "13000,0",
"direct_debit_interval": 1,
"direct_debit_amount": "13000,0",
"bank_account_bank_name": "First Bank of Donuts",
"bank_account_holder": "Thomas Coram",
"bank_account_iban": "12332984",
"bank_account_bic": "12000",
"pdf":
https://app.formunauts.com/api/v2/donations/pdf/?uid=2028212078,
"contract_start_date": "2015-05-30",
"confirm_information_check": true,
"confirm_fundraiser_incentive_information": true,
"welcome_email_status": "queued",
"comments_qa": "",
"location": "London - South East (SE1)",
"sign_up_coordinates": [0.053695, 51.474973],
"confirm_direct_debit": true,
"confirm_bank_account": true,
"confirm_satisfied": true,
"special1": "",
"special2": "",
"comments": "Great guy!",

```

```

        "createtime": "2015-05-27T08:17:43.490Z",
        "uploadtime": "2015-05-27T08:17:43.490Z",
        "feedback_rating": 3.0,
        "status": "held",
        "status_comment": "Please check back with donor on
address",
        "status_reason": "FVO-10-001",
        "external_status_reason": "",
        "payment_method": "donut-sepa",
        "original_payment_method": "donut-sepa",
        "quality_check_passed": false,
        "change_note_private": "",
        "change_note_public": "",
        "raisenow_epp_transaction_id": "als2df3",
        "raisenow_subscription_id": "2df3als",
        "raisenow_status": "final_success",
        "raisenow_subscription_canceled": false,
        "raisenow_subscription_uuid":
        "ee73846d-662d-46eb-8ff4-1fac868b61ea",
        "raisenow_payment_uuid":
        "6297ddd7-4b35-476a-a51a-a405bcb11012",
        "amount_covered": "2,50",
        "stripe_customer_id": "9u8s7e6g5z4i3l2o1n",
        "team_tags": ["some-tag", ...],
        "team_id": 1908,
        "tax_identification_number": "12345"
        [additional fields]
    },
    { ... }
]
}

```

Explanation of the JSON fields:

uid	Integer	Unique ID of the donation
person_id	String	Person ID
organisation_id	String	Organisation ID
agency_id	String	Agency ID

customer_id	Integer	Unique ID of the agency the donation was collected for
campaign_id	Integer	Unique ID of the campaign the donation was collected for
campaign_type	Integer	0 -> door2door 1 -> info booth 2 -> private site 3 -> event null -> not set
campaign_type2	String	either "week_campaign" or "city_campaign"
donor_first_name	String	First name of donor
donor_last_name	String	Last name of donor
donor_company_name	String	Company name
donor_academic_title	String	Academic title of donor as string value
donor_salutation	Integer	1 -> "Herr" / "Mr" 2 -> "Frau" / "Mrs" 3 -> "Familie" / "Family" 4 -> "Firma" / "Company" 5 -> "Sonstiges" / "Other" 6 -> "Ms" 7 -> "Miss" 9 -> "Dr" null -> not set
donor_sex	Integer	1 -> male 2 -> female 3 -> family
donor_date_of_birth	String	Date of birth in ISO format YYYY-MM-DD
donor_age_in_years	Integer	Numerical value, age at sign up date.
donor_occupation	Integer	1 -> Worker 2 -> Employee 3 -> Retiree 4 -> Self employed 5 -> Student 6 -> Other 17 -> Sole Trader 18 -> Home Maker

		37 -> Carer
donor_email	String	Email address of donor
donor_phone	String	Phone number of donor
donor_mobile	String	Mobile phone number of donor
donor_zip_code	String	Zip code
donor_street	String	Street name
donor_house_number	String	House number
donor_city	String	City name
donor_department	String	Department name
donor_province	String	Province name
donor_district	String	District name
donor_address_addition	String	Address Addition if field out
donor_address_addition_2	String	Second address addition field if field out
donor_country	String	Two letter ISO country code like: AT, DE, UK
contact_by_email	Integer	If the donor wants to be contacted by email 0 -> no 1 -> yes
contact_by_phone	Integer	If the donor wants to be contacted by phone 0 -> no 1 -> yes
fundraiser_code	String	Identification code of fundraiser
fundraiser_name	String	Name of fundraiser in format "last name, first name"
fundraiser_id	Integer	Reference to fundraiser can be used in Fundraiser List endpoint to get more data
fundraiser_external_id	String	String, used as reference in external systems (eg. CRMs)
donation_amount_annual	String	Annual donation. Formatted float value with "," (comma) as the floating point character

direct_debit_amount	String	Amount per chosen payment interval. Formatted float value with “,” (comma) as the floating point character
direct_debit_interval	Integer	1 -> annual 2 -> biannual 4 -> quarterly 12 -> monthly 0 -> single payment
bank_account_bank_name	String	Name of the Bank entered on the form, else empty string
bank_account_holder	String	Name of the bank account holder
bank_account_iban	String	IBAN
bank_account_bic	String	BIC
pdf	String	Link to PDF of donation with signature of donor. (see Downloading PDFs section)
special1	String	alphanumeric codes in CSV format
special2	String	alphanumeric codes in CSV format
comments	String	Comments
createtime	String	Creation date and time of the donation in ISO 8601 format. Example: “2015-05-27T08:17:43.490Z”
uploadtime	String	Upload date and time of the donation in ISO 8601 format.
contract_start_date	String	Start date of the contract in ISO format YYYY-MM-DD. Can be null
location	String	Name (and optional location code in parenthesis) of the location where the donation was collected. Example: “Hauptplatz” or “Hauptplatz (001)”
sign_up_coordinates	List of floats	The geographical coordinates (in degrees, longitude first, latitude second) where the donor was signed up. Can be null if unknown.
welcome_email_status	String	Status of the welcome mail. Can be one of

		these or null: queued -> email was queued for sending sent -> successfully sent open -> the donor has opened the email failed -> email was not sent bounce -> email was not delivered because the email bounced in the target inbox hard_bounce -> email was not delivered because of hard bounce in the target inbox (permanent) blocked -> email was not delivered because the sending was blocked by mailjet retrying -> we retry to send the email after a failed attempt click -> the recipient has clicked on content in the email spam -> the email was put into the spam folder of the recipient deferred -> the receiving server delayed acceptance of the message
comments_qa	String	Comments from the quality assessment form. Can be null NOTE: since release 1.12 of the API this field is included in the additional fields and depends on campaign configuration. (see Flexible Fields)gg
confirm_direct_debit*	Bool	Answer to the statement “I was told that my support is in the form of a recurring payment that will be withdrawn from my bank account.” on the quality assessment form. Can be true, false or null NOTE: since release 1.12 of the API this field is included in the additional fields and depends on campaign configuration. (see Flexible Fields)
confirm_bank_account*	Bool	Answer to the statement “My bank account information was verified with my bank card.” on the quality assessment form. Can be true, false or null

		NOTE: since release 1.12 of the API this field is included in the additional fields and depends on campaign configuration. (see Flexible Fields)
confirm_satisfied*	Bool	Answer to the question “Were you satisfied with the conversation in general?” on the quality assessment form. Can be true, false or null NOTE: since release 1.12 of the API this field is included in the additional fields and depends on campaign configuration. (see Flexible Fields)
confirm_information_check*	Bool	Answer to the question “Have you verified the information you supplied?” on the quality assessment form. Can be true, false or null NOTE: since release 1.12 of the API this field is included in the additional fields and depends on campaign configuration. (see Flexible Fields)
confirm_fundraiser_incentive_information*	Bool	Answer to the question “Our fundraisers are paid a guaranteed fee plus bonuses for performance and quality. Have you been information about this?” on the assessment form. Can be true, false or null NOTE: since release 1.12 of the API this field is included in the additional fields and depends on campaign configuration. (see Flexible Fields)
feedback_rating	Float	Float Value between 1.0 (worst) and 5.0 (best)
feedback_rating_custom_answer	String	Free text part of Feedback Form on Quality Widget
status	String	Donation status according to list at Appendix A: Status Codes (only “none” and “held” are available if dataflow is not enabled)

status_comment	String	Freertext note to aid confirmation process
status_reason	String	status reason according to list at Appendix B: Status Reasons only available if workflow is enabled for the campaign.
external_status_reason	String	status reason deriving from import from external source (only if workflow is enabled for campaign)
payment_method	String	identifier of donations payment method. currently (2025-07-01, Version 2.8) available options are: “apple_pay_tamara” - “Apple Pay over RaiseNow using Tamara Widget” “cash” - Cash Donation (no Bank data) “cbu” - Bank Payment in Argentina “cc_mpos” - Credit Card payment through mpos device on old payment platform “cc_raisenow” - credit card payment on RaiseNow’s old payment platform “cc_tamara” - Credit Card over RaiseNow using Tamara Widget “dd” - direct debit via RaiseNow “dd_tamara” - Direct Debit over RaiseNow using Tamara Widget “donut-de-kontonummer” - old Style Bank Account Numbers “donut-sepa” - SEPA payments “donut-sepa-eur” - SEPA payments in switzerland “donut-uk-sort-codes” - UK Bank account using sortcodes “google_pay_tamara” - Google Pay over RaiseNow using Tamara Widget “payku” - payment provider in Chile “payment_slip” - Payment slip “paypal_tamara” - PayPal over RaiseNow using Tamara widget “postfinance_iban” - PostFinanz payment via IBAN “self-checkout” - payment via self checkout on donors device “skip” - payment info omitted (only valid for unfinished donations)

		"swiss_direct_debit" - direct debit payment in switzerland "twint_tamaro" - TWINT over RaiseNow using Tamaro Widget "wallet_tamaro" - Wallet payment over RaiseNow using Tamaro Widget
original_payment_method	String	Available in workflow enabled campaignas where <code>payment_method</code> is subject to change via imports. values equal the <code>payment_method</code> field.
quality_check_passed	Bool	indicates if the donation passed the quality checks. available if quality checks are enabled on the campaign.
change_note_private	String	Internal change note
change_note_public	String	Change note displayed to donor in email
raisenow_epp_transaction_id	String	Alphanumeric transaction ID from raisenow connector, if used. Else always null - This field will be deleted once RaiseNow fully migrated to hub and the old manager reached EOL (currently date is set to 2025-09-30)
raisenow_subscription_id	String	Alphanumeric subscription ID from raisenow connector, if used. Else an empty string. - This field will be deleted once RaiseNow fully migrated to hub and the old manager reached EOL (currently date is set to 2025-09-30)
raisenow_status	String	Status of RaiseNow payment: • possible values for (old) EPP payments: <code>"final_success"</code>, <code>"final_error"</code>, <code>"aborted_by_user"</code> • possible values for (new) EPMS payments <code>"active"</code>, <code>"pending"</code>, <code>"suspended"</code>, <code>"cancelled"</code>
raisenow_subscription_canceled	Bool	Flag set if the subscription was canceled by the donor - this field is only displayed for old EPP

		payments, in EPMS payments it is replaced with <code>raisenow_status = 'cancelled'</code>
<code>raisenow_subscription_uuid</code>	String	field for identifying subscriptions in new RaiseNow hub - only added for Donations with EPMS payment
<code>raisenow_payment_uuid</code>	String	field for identifying payments in new RaiseNow hub - only added for Donations with EPMS payment
<code>amount_covered</code>	String	Amount chosen in 'Cover the Fee' feature in Cents - only added for Donations with EPMS payment Formatted float value with "," (comma) as the floating point character. Will be displayed as "0,00" if not set.
<code>stripe_customer_id</code>	String	Alphanumeric ID for the stripe customer. Null in case of a non stripe payment.
<code>team_tags</code>	List	All tags from the team as List of Strings
<code>team_id</code>	Integer	id of the team as reference in the team lists endpoint
<code>tax_identification_number</code>	String	Tax identification number, if available

NEW in version 1.18:

An optional boolean field `contact_data_conf` (possible values: `true`, `false`, `null`) was added to the donations that is only included if the campaign is configured to include the field on the form.

NEW in version 1.25:

If importing of retention data is enabled, there is an array holding the `payment_history` added to the Donation response.

Example response:

```
{
```

```

.../
"payment_history": [
  {
    "date": "2018-08-20",
    "type": "payment",
    "status": "",
    "successful_payment_count": 1,
    "gross_payment_count": 1,
    "payment_file": "payment_2018082"
  },
  ...
]
}

```

Explanation of the JSON Fields:

date	String	Date of payment operation according to payment file
type	String	Either “payment” or “cancellation”
status	String	Additional Status code according to payment file (string value)
successfull_payment_count	Integer	Count of total successful payment
gross_payment_count	Integer	Count of total payment attempts (no matter if successful or not)
payment_file	String	Name of import file

NEW in version 1.26:

It is now possible to choose between multiple currencies on the donation form. If the campaign has multiple currencies configured the following fields are added to the Donation response:

<code>donation_amount_annual_in_original_currency</code>	String	If the currency selected in the form is different from the default currency of the campaign this is the annual donation amount. Formatted float value with “,” (comma) as the floating point character
<code>donation_original_currency</code>	String	If the currency selected in the form is different from the default currency of the campaign this is the currency that was selected. Three letter uppercase ISO 4217 currency code.

NEW in version 1.31:

Renamed previously undocumented field `sextant_vendor_id` to `fundraiser_external_id` and added to documentation.

NEW in version 2.1:

Added 2 optional fields for campaigns with active self-checkout configuration

<code>sco_subscription_active</code>	Bool	status of the sco process subscription. <code>False</code> if user unsubscribed from the SCO process, else <code>True</code>
<code>sco_journey_completed</code>	String	Depending on the SCO journey either “sco”, “sco_payment” or null if the donation was not using SCO, or is still unconfirmed.

NEW in version 2.2:

If `dataflow` is enabled on the campaign of the donation following fields that are filled by the callcenter import will be added to the response

welcome_call_outcome	String	status reason code outcome of the welcome call. see list in Appendix B: Status Reasons
welcome_call_attempts	Integer	count of welcome call attempts
welcome_call_first_attempt	String	Date and time of first welcome call attempt in ISO 8601 format
welcome_call_latest_attempt	String	Date and time of latest welcome call attempt in ISO 8601 format
welcome_call_comment	String	free text field
welcome_call_inbound_number	String	Inbound number
welcome_call_public_change_note	String	free text field
welcome_call_internal_change_note	String	free text field
welcome_call_qq_face_covering	Bool	Quality Question - Face Covering
welcome_call_qq_id_badge	Bool	Quality Question - ID Badge
welcome_call_qq_branded_clothing	Bool	Quality Question - Branded Clothing
welcome_call_qq_paid_fundraiser	Bool	Quality Question - Paid Fundraiser
welcome_call_qq_opt_ins_confirmed	Bool	Quality Question - Opt ins confirmed (changed in Version 2.7)
welcome_call_qq_fundraiser_benefits	Bool	Quality Question - Fundraiser Benefits

NEW in version 2.5:

In version 2.5 we added `?date_from=yyyy-mm-dd` and `?date_to=yyyy-mm-dd` filters to the donation endpoint. See [Filtering](#) section for more details.

NEW in version 2.8:

Added fields `raisenow_subscription_uuid`, `raisenow_payment_uuid` for connection to new RaiseNow hub, as well as `amount_covered` set in the new 'Cover the Fee' Feature. All new fields are available for Donations using epms payment methods. Marked fields connected to old Raisenow Manager (`raisenow_epp_transaction_id`, `raisenow_subscription_id`, `raisenow_subscription_canceled`) for deprecation.

NEW in version 2.9:

Added field `sign_up_coordinates`.

Confirm retrieval of donations

After you have fetched the donations with the request above and have also downloaded the PDFs of the donations, you have to confirm that you have received the donations. This is done by sending an array of donation uids to this endpoint. As a response you get an array of objects. Every object represents the status of the confirmation of one donation.

URL: `/donations/confirm_retrieval/`

Method: POST

Data:

This request expects you to post an array of JSON objects. The post data must have the `application/json` content-type.

The array must have the following form:

```
[
  {
    "uid": 2602215248,
    "status": "success",
    "message": ""
  },
  {
    "uid": 3554067970,
```

```

        "status": "error",
        "message": "Could not download PDF."
    },
    { ... }
]

```

If you give a `status` of “success” the donation will be marked as retrieved and will not be included in the response of the `/donations/` request anymore. The PDF with the signature of the donation will also be deleted on our servers. The “message” field is ignored when status is “success”.

If you give a status of “error” you can give an error message to the server. This error message will be logged. The donation will continue to be listed in the donations returned by the `/donations/` request. The PDF of the donation will also still be available for download.

Example request:

```

curl -i -X POST -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" -d '[{"uid": 2602215248,"status": "success","message": ""}, {"uid": 3554067970,"status": "error","message": "Could not download PDF."}, {"uid": 123123123,"status": "success","message": ""}]'
https://app.formunauts.com/api/v2/donations/confirm_retrieval/

```

Example response:

```

[
  {
    "status": "success",
    "message": "",
    "uid": 2602215248,
    "confirmation_date": "2015-05-28T15:10:54.921"
  },
  {
    "status": "error",
    "message": "Could not download PDF.",
    "uid": 3554067970
  },
  {
    "status": "error",
    "message": "Diese Spende existiert nicht.",
    "uid": 123123123
  }
]

```

]

For every donation you get a status if the confirmation was successful. If the confirmation was successful you get the date and time of the confirmation. In case it was not successful you get an additional error message.

IMPORTANT NOTICE

Because of performance considerations we do not recommend confirming more than 100 donations at once.

List unfinished donations

URL: /donations/unfinished/

Method: GET

Example request:

```
curl -i -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" https://app.formunauts.com/api/v2/donations/unfinished/
```

Example response:

See [list donations](#). Fields that do not apply to unfinished donations (welcome_email_status, pdf, person_id) are delivered with null values.

NEW in version 2.5:

In version 2.5 we added ?date_from=yyyy-mm-dd and ?date_to=yyyy-mm-dd filters to the unfinisheddonations endpoint. See Filtering section for more details.

Confirm retrieval of unfinished donations

After you have fetched the unfinished donations with the request above, you have to confirm that you have received the donations. This is done by sending an array of unfinished donation uids to this endpoint. As a response you get an array of objects. Every object represents the status of the confirmation of one unfinished donation.

URL: /donations/unfinished/confirm_retrieval/

Method: POST

Data:

This request expects you to post an array of JSON objects. The post data must have the application/json content-type.

The array must have the following form:

```
[
  {
    "uid": 2602215248,
    "status": "success",
    "message": ""
  },
  {
    "uid": 3554067970,
    "status": "error",
    "message": "Could not download PDF."
  },
  { ... }
]
```

If you give a status of “success” the donation will be marked as retrieved and will not be included in the response of the /donations/unfinished/ request anymore. The “message” field is ignored when status is “success”.

If you give a status of “error” you can give an error message to the server. This error message will be logged. The donation will continue to be listed in the donations returned by the /donations/unfinished/ request.

Example request:

```
curl -i -X POST -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer
```

```
ZoGvK0RBdJnqxoItlqT9KnygmYek3v" -d '[{"uid": 2602215248,"status":
"success","message": ""},{ "uid": 3554067970,"status":
"error","message": "Could not download PDF."},{ "uid":
123123123,"status": "success","message": ""}]'
https://app.formunauts.com/api/v2/donations/unfinished/confirm_retrie
val/
```

Example response:

```
[
  {
    "status": "success",
    "message": "",
    "uid": 2602215248,
    "confirmation_date": "2015-05-28T15:10:54.921"
  },
  {
    "status": "error",
    "message": "Could not download PDF.",
    "uid": 3554067970
  },
  {
    "status": "error",
    "message": "Diese Spende existiert nicht.",
    "uid": 123123123
  }
]
```

For every unfinished donation you get a status if the confirmation was successful. If the confirmation was successful you get the date and time of the confirmation. In case it was not successful you get an additional error message. Because of performance considerations we do not recommend confirming more than 100 unfinished donations at once.

List donation feedback

Returns a list of donation feedbacks ready to be fetched via the API. Information about Donation Feedbacks is also included in the [List donations](#) endpoint, but differs in the data format for backwards compatibility reasons.

URL: /donations/feedback/

Method: GET

Example request:

```
curl -i -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" https://app.formunauts.com/api/v2/donations/feedback/
```

Example response:

```
{
  "count": 37,
  "total_pages": 4,
  "next": "https://app.formunauts.com/api/v2/donations/feedback/?page=2",
  "previous": null,
  "results": [{
    "uid": 1345678901,
    "fundraiser_id": 47,
    "donation_uid": 2028212078,
    "createtime": "2021-03-03T11:11:43.814Z",
    "rating": "3.00",
    "compliments": ["great_conversation", "made_my_day"],
    "comment": "Nice dude!",
    "comment_approved": true,
    "quality_feedback": ["no_id", "too_fast"],
  },
  {...}
]
```

Explanation of the JSON fields:

uid	Integer	Unique ID of the donation feedback
fundraiser_id	Integer	ID of fundraiser retrieving the feedback
donation_uid	Integer	UID of the linked donation
createtime	String	Creation date and time of the fundraiser entry in ISO 8601 format.
rating	Decimal	Decimal value between 1.0 and 5.0

compliments	Array	Array of predefined compliments. Possible values: "great_conversation", "made_my_day", "very_informative", "good_job", "inspiring"
comment	String	Optional free text comment of the donor
comment_approved	Boolean	comment was approved by admin (can be null if not viewed by admin yet)
quality_feedback	Array	Array of predefined quality feedbacks. Possible values: "no_id", "too_fast", "no_clue"

Confirm retrieval of Donation Feedback

After you have fetched the donation feedback with the request above, you have to confirm that you have received the donations. This is done by sending an array of donation feedback ids to this endpoint. As a response you get an array of objects. Every object represents the status of the confirmation of one donation feedback.

URL: /donations/feedback/confirm_retrieval/

Method: POST

Data:

This request expects you to post an array of JSON objects. The post data must have the application/json content-type.

The array must have the following form:

```
[
  {
    "uid": 1345678901,
    "status": "success",
    "message": ""
  },
  {
    "uid": 1345678902,
    "status": "error",
    "message": "Could not find linked donation"
  },
  { ... }
```


]

If you give a `status` of “success” the donation feedback will be marked as retrieved and will not be included in the response of the `/donations/feedback/` request anymore. The “message” field is ignored when status is “success”.

If you give a status of “error” you can give an error message to the server. This error message will be logged. The donation will continue to be listed in the donations returned by the `/donations/feedback/` request.

Example request:

```
curl -i -X POST -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" -d '[{"uid": 1345678901,"status": "success","message": ""},{ "uid": 1345678902,"status": "error","message": "Could not find linked donation"}]' https://app.formunauts.com/api/v2/donations/feedback/confirm_retrieval/
```

Example response:

```
[
  {
    "status": "success",
    "message": "",
    "uid": 1345678901,
    "confirmation_date": "2021-03-03T11:29:03.903Z"
  },
  {
    "status": "error",
    "message": "Could not find linked donation",
    "uid": 1345678902
  },
]
```

For every donation feedback you get a status if the confirmation was successful. If the confirmation was successful you get the date and time of the confirmation. In case it was not successful you get an additional error message. Because of performance considerations we do not recommend confirming more than 100 donation feedbacks at once.

List petitions

Returns a list of the petitions in the system that are ready to be fetched via the API.

URL: /petitions/

Method: GET

Example request:

```
curl -i -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" https://app.formunauts.com/api/v2/petitions/
```

Example response:

```
{
  "count": 33,
  "total_pages": 4,
  "next": "https://app.formunauts.com/api/v2/petitions/?page=2",
  "previous": null,
  "results": [{
    "uid": 1837446972,
    "person_id": "STW8000500310",
    "organisation_id": "333333",
    "agency_id": "",
    "customer_id": 1,
    "campaign_id": 6,
    "campaign_type": 1,
    "campaign_type2": "city_campaign",
    "donor_first_name": "Margarethe",
    "donor_last_name": "Stöckler",
    "donor_academic_title": null,
    "donor_salutation": 2,
    "donor_sex": 2,
    "donor_date_of_birth": "1955-11-19",
    "donor_age_in_years": 63,
    "donor_occupation": 6,
    "donor_email": "margarethe@gmail.com",
    "donor_phone": "+43112312322",
    "donor_mobile": "",
```

```

        "donor_zip_code": "1160",
        "donor_street": "Eisenstadtplatz",
        "donor_house_number": "4/3/4",
        "donor_city": "Wien",
        "donor_department": "",
        "donor_province": "Wien",
        "donor_district": "",
        "donor_address_addition": "",
        "donor_address_addition_2": "",
        "donor_country": "AT",
        "contact_by_email": 0,
        "contact_by_phone": 0,
        "fundraiser_code": "d-00789",
        "fundraiser_name": "Donut, James",
        "fundraiser_id": 47,
        "fundraiser_external_id": "ext-186019081899",
        "team_id": 1908,
        "comments": null,
        "pdf":
"https://app.formunauts.com/api/v2/petitions/pdf/?uid=1837446972",
        "special1": "",
        "special2": "",
        "createtime": "2015-12-16T17:50:01.618Z",
        "uploadtime": "2015-12-16T17:50:01.618Z",
        "change_note_private": "",
        "welcome_email_status": "sent",
        "status": "none",
        "status_comment": "",
        [additional fields]
    },
    { ... }
]
}

```

Explanation of the JSON fields:

uid	Integer	Unique ID of the petition
person_id	String	Person ID
organisation_id	String	Organisation ID

agency_id	String	Agency ID
customer_id	Integer	Unique ID of the agency the petition was collected for
campaign_id	Integer	Unique ID of the campaign the petition was collected for
campaign_type	Integer	0 -> door2door 1 -> info booth 2 -> private site 3 -> event null -> not set
campaign_type2	String	either "week_campaign" or "city_campaign"
donor_first_name	String	First name of donor
donor_last_name	String	Last name of donor
donor_academic_title	String	Academic title of donor
donor_salutation	Integer	null -> <i>nothing selected</i> 1 -> "Herr" / "Mr" 2 -> "Frau" / "Mrs" 3 -> "Familie" / "Family" 4 -> "Firma" / "Company" 5 -> "Sonstiges" / "Other" 6 -> "Ms" 7 -> "Miss" 9 -> "Dr"
donor_sex	Integer	1 -> male 2 -> female 3 -> family
donor_date_of_birth	String	Date of birth in ISO format YYYY-MM-DD
donor_age_in_years	String	Numerical value, age at sign up date.
donor_occupation	Integer	null -> <i>nothing selected</i> 1 -> Worker 2 -> Employee 3 -> Retiree 4 -> Self employed 5 -> Student 6 -> Other 17 -> Sole Trader

		18 -> Home Maker
donor_email	String	Email address of donor
donor_phone	String	Phone number of donor
donor_mobile	String	Mobile phone number of donor
donor_zip_code	String	Zip code
donor_street	String	Street name
donor_house_number	String	House number
donor_city	String	City name
donor_department	String	Department name
donor_province	String	Province name
donor_district	String	District name
donor_address_addition	String	Address Addition if field out
donor_address_addition_2	String	Second address addition field if field out
contact_by_phone	Integer	If the donor wants to be contacted by phone 0 -> no 1 -> yes
fundraiser_code	String	Identification code of fundraiser
fundraiser_name	String	Name of fundraiser in format "last name, first name"
fundraiser_id	Integer	Reference to fundraiser can be used in Fundraiser List endpoint to get more data
fundraiser_external_id	String	String, used as reference in external systems (eg. CRMs)
team_id	Integer	id of the team as reference in the team lists endpoint
pdf	String	Link to PDF of petition with signature of supporter. (see Downloading PDFs section)
special1	String	alphanumeric codes in CSV format
special2	String	alphanumeric codes in CSV format

comments	String	Comments
createtime	String	Creation date and time of the petition in ISO 8601 format. Example: "2015-05-27T08:17:43.490Z"
uploadtime	String	Upload date and time of the petition in ISO 8601 format.
welcome_email_status	String	Status of the welcome mail. Can be one of these or null: queued -> email was queued for sending sent -> successfully sent open -> the donor has opened the email failed -> email was not sent bounce -> email was not delivered because the email bounced in the target inbox blocked -> email was not delivered because the sending was blocked by mailjet retrying -> we retry to send the email after a failed attempt click -> the recipient has clicked on content in the email spam -> the email was put into the spam folder of the recipient deferred -> the receiving server delayed acceptance of the message
change_note_private	String	Internal change note
change_note_public	String	Change note displayed to donor in email
status	String	Donation status according to list at https://formunauts.atlassian.net/wiki/spaces/DOC/pages/2237923331/Donation+Status+Reasons#Status
status_comment	String	Freetext note to aid confirmation process
contact_data_conf	Bool	Confirmation of concat on form, shown only if the campaign has the field enabled on the form.

Confirm retrieval of petitions

After you have fetched the petitions with the request above and have also downloaded the PDFs of the petitions, you have to confirm that you have received the petitions. This is done by

sending an array of petition uids to this endpoint. As a response you get an array of objects. Every object represents the status of the confirmation of one petition.

URL: /petitions/confirm_retrieval/

Method: POST

Data:

This request expects you to post an array of JSON objects. The post data must have the application/json content-type.

The array must have the following form:

```
[
  {
    "uid": 1837446972,
    "status": "success",
    "message": ""
  },
  {
    "uid": 1837446973,
    "status": "error",
    "message": "Could not download PDF."
  },
  { ... }
]
```

If you give a status of “success” the petition will be marked as retrieved and will not be included in the response of the /petitions/ request anymore. The PDF with the signature of the petition will also be deleted. The “message” field is ignored when status is “success”.

If you give a status of “error” you can give an error message to the server. This error message will be logged. The petition will continue to be listed in the petitions returned by the /petitions/ request. The PDF of the petition will also still be available for download.

Example request:

```
curl -i -X POST -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" -d '[{"uid": 1837446972,"status": "success","message": ""}, {"uid": 1837446973,"status": "error","message": "Could not download PDF."}]' https://app.formunauts.com/api/v2/petitions/confirm_retrieval/
```


Example response:

```
[
  {
    "status": "success",
    "message": "",
    "uid": 1837446972,
    "confirmation_date": "2016-01-04T13:13:54.921"
  },
  {
    "status": "error",
    "message": "Could not download PDF.",
    "uid": 1837446973
  }
]
```

For every petition you get a status if the confirmation was successful. If the confirmation was successful you get the date and time of the confirmation. In case it was not successful you get an additional error message. Because of performance considerations we do not recommend confirming more than 100 petitions at once.

NEW in version 2.5:

In version 2.5 we added `?date_from=yyyy-mm-dd` and `?date_to=yyyy-mm-dd` filters to the petitions endpoint. See [Filtering](#) section for more details.

List time tracking entries

Returns a list of the time tracking entries in the system that are ready to be fetched via the API.

URL: `/timeentries/`

Method: GET

Example request:

```
curl -i -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" https://app.formunauts.com/api/v2/timeentries/
```

Example response:

```
{
  "count": 233,
  "total_pages": 24,
  "next":
    "https://app.formunauts.com/api/v2/timeentries/?page=3",
  "previous":
    "https://app.formunauts.com/api/v2/timeentries/?page=1",
  "results": [{
    "uid": 1432300054,
    "organisation_id": "123121",
    "agency_id": "123131",
    "customer_id": 6,
    "campaign_id": 21,
    "fundraiser_code": "r-007",
    "coach_code": "r-081",
    "teamleader_code": "r-7777",
    "campaign_type": 1,
    "start_time_shift": "2015-07-02T07:45:02",
    "end_time_shift": "2015-07-02T17:45:09",
    "breaks": [{
      "end_break": "2015-07-02T16:15:00",
      "start_break": "2015-07-02T16:00:00"
    },
    {
      "end_break": "2015-07-02T14:00:00",
      "start_break": "2015-07-02T13:40:00"
    },
    {
      "end_break": "2015-07-02T11:45:03",
      "start_break": "2015-07-02T11:14:02"
    }
  ],
  "num_breaks": 3,
  "work_hours": 9.42,
  "break_hours": 0.58
  "createtime": "2015-06-01T09:02:58.837Z"
},
{ ... }
]
```

Explanation of the JSON fields:

uid	Integer	Unique ID of the time tracking entry
organisation_id	String	Organisation ID for DaDi
agency_id	String	Agency ID for DaDi
customer_id	Integer	Unique ID of the agency the time tracking entries was collected for.
campaign_id	Integer	Unique ID of the campaign the time tracking entry was collected for.
campaign_type	Integer	0 -> door2door 1 -> info booth 2 -> private site 3 -> event null -> not set
start_time_shift	String	Start time of the work shift in ISO 8601 format. Example: "2015-06-01T11:02:58"
breaks	List	Array of objects that represent a work break.
num_breaks	Integer	The number of breaks in the array.
work_hours	Float	The number of hours the person has worked (does not include the break times).
break_hours	Float	The number of hours the person was on break.
end_time_shift	String	End time of the work shift in ISO 8601 format. Example: "2015-06-01T11:02:58"
createtime	String	Creation date and time of the time tracking entry in ISO 8601 format. Example: "2015-05-27T08:17:43.490Z"

Confirm retrieval of time tracking entries

After you have fetched the time tracking entries with the request above, you have to confirm that you have received the time tracking entries. This is done by sending an array of time tracking entry uids to this endpoint. As a response you get an array of objects. Every object represents the status of the confirmation of one time tracking entry.

URL: /timeentries/confirm_retrieval/

Method: POST

Data:

This request expects you to post an array of JSON objects. The post data must have the application/json content-type.

The array must have the following form:

```
[
  {
    "uid": 1432300054,
    "status": "success",
    "message": ""
  },
  {
    "uid": 1432312354,
    "status": "error",
    "message": "Some error message."
  },
  {
    "uid": 1432312366,
    "status": "success",
    "message": ""
  }
]
```

If you give a status of “success” the time tracking entry will be marked as retrieved and will not be included in the response of the /timeentries/ request anymore. The “message” field is ignored when status is “success”.

If you give a status of “error” you can give an error message to the server. This error message will be logged. The time tracking entry will continue to be listed in the time tracking entries returned by the /timeentries/ request.

Example request:

```
curl -i -X POST -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" -d '[{"uid": 1432300054,"status": "success","message": ""},{ "uid": 1432312354,"status": "error","message": "Some error message."},{ "uid": 1432312366,"status":
```

```
"success", "message": ""}]'
https://app.formunauts.com/api/v2/timeentries/confirm_retrieval/
```

Example response:

```
[
  {
    "status": "success",
    "message": "",
    "uid": 2602215248,
    "confirmation_date": "2015-05-28T15:10:54.921"
  },
  {
    "status": "error",
    "message": "Some error message.",
    "uid": 3554067970
  },
  { ... }
]
```

For every time tracking entry you get a status if the confirmation was successful. If the confirmation was successful you get the date and time of the confirmation. In case it was not successful you get an additional error message. Because of performance considerations we do not recommend confirming more than 100 time tracking entries at once.

List fundraisers

Returns a list of fundraisers. Add id to request to query a single fundraiser.

URL: /fundraisers/[fundraiser_id]/

Method: GET

Filters: the list of fundraisers can optionally be filtered by fundraiser status by appending ?is_active=true or ?is_active=false to the URL

Example request:

```
curl -i -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" https://app.formunauts.com/api/v2/fundraisers/
```

Example response:

```
{
  "count": 12,
  "total_pages": 2,
  "next": "https://app.formunauts.com/api/v2/fundraisers/?page=2",
  "previous": null,
  "results": [{
    "id": 47,
    "first_name": "Alex",
    "last_name": "Schwaiger",
    "street": "",
    "zip_code": "",
    "city": "",
    "email": "dev@formunauts.at",
    "phone_number": "06761231233",
    "fundraiser_code": "002",
    "image_upload": "https://assets.formunauts.com/media/uploads/fundraiser_photos/f_94ed3002aa2d41f79e77897cc3fb823d.jpg",
    "is_teamleader": false,
    "is_coach": false,
    "is_active": false,
    "customer": 6
  ]
}
```

```

        "createtime": "2014-05-21T07:16:59.923335Z",
        "modifytime": "2015-11-02T13:23:12.998567Z",
        "fundraiser_external_id": "ext-186019081899",
        "username": "demo-002",
        "language": "de",
        "staffcloud_refid": 899,
        "staffcloud_import_date": "2019-09-28T01:00:15.978449Z",
        "image_thumbnail_url":
"https://assets.formunauts.com/media/cache/58/e0/58e001005341fd3985374b0d9da526a4.png",
    },
    {
        "id": 422,
        "first_name": "James",
        "last_name": "Bond",
        "street": "",
        "zip_code": "",
        "city": "",
        "email": "dev@formunauts.at",
        "phone_number": "",
        "fundraiser_code": "007",
        "image_upload":
"https://assets.formunauts.com/media/uploads/fundraiser\_photos/f\_94ed3002aa2d41f79e77897cc3fb823d.jpg",
        "is_teamleader": true,
        "is_coach": true,
        "is_active": true,
        "customer": 6,
        "createtime": "2015-02-04T18:08:32.607390Z",
        "modifytime": "2016-04-11T08:02:32.047467Z",
        "fundraiser_external_id": "ext-0071969",
        "username": "demo-007",
        "language": "en",
        "staffcloud_refid": 887,
        "staffcloud_import_date": "2019-09-28T01:00:15.978449Z",
        "image_thumbnail_url":
"https://assets.formunauts.com/media/cache/58/e0/58e001005341fd3985374b0d9da526a4.png",
    }
}

```

Explanation of the JSON fields:

id	Integer	readonly	Internal unique ID
first_name	String	writable required	First name of fundraiser
last_name	String	writable	Last name of fundraiser
street	String	writable	Street and house number
zip_code	String	writable	Zip code / Postal code
email	String	writable	E-mail address of fundraiser
phone_number	String	writable	Phone number (most times the mobile phone number)
fundraiser_code	String	writable required	Fundraiser code used in login
image_upload	String	writable	URL of original upload of fundraiser image
image_thumbnail_url	String	readonly	URL of the Fundraiser Avatar Image (cropped)
is_teamleader	Boolean	writable	Flag indicating if the fundraiser has team leader role
is_coach	Boolean	writable	Flag indicating if the fundraiser has coach role
is_active	Boolean	writable	Flag indicating if the fundraiser is active. Fundraisers are not deleted, they are set to inactive.
customer	Integer	readonly	The internal unique ID of the customer this fundraiser is associated with
createtime	String	readonly	Creation date and time of the fundraiser entry in ISO 8601 format.

modifytime	String	readonly	Last modification date and time of the time tracking entry in ISO 8601 format.
fundraiser_external_id	String	writable	used as reference in external systems (eg. CRMs)
username	String	readonly	the username used for login, usually the customer login prefix + the fundraiser_code
language	String	writable	ISO 639-1 language code of the fundraisers main language
staffcloud_refid	Integer	readonly	ID in Staffcloud If imported from there
staffcloud_import_date	String	readonly	Date of import from Staffcloud

Add a new fundraiser without fundraiser image

Adds a new fundraiser. If successful a HTTP status code 201 CREATED is returned.

URL: /fundraisers/

Method: POST

Data:

This request expects you to post an JSON object. The post data must have the application/json content-type.

Here is an example for a possible JSON:

```
{
  "first_name": "Lisi",
  "last_name": "Schreiber",
  "street": "Webgasse 4",
  "zip_code": "1030",
  "city": "Wien",
  "email": "test@test.com",
  "phone_number": "+43699123123213",
  "fundraiser_code": "0888",
  "customer": 6,
```

```
    "is_active": 1,  
  }
```

For the full list of writable fields please refer to the ['List fundraisers'](#) section.

Example request:

```
curl -i -X POST -H "Content-Type: application/json" -H "Accept:  
application/json" -H "Authorization: Bearer  
ZoGvK0RBdJnqxoItlqT9KnygmYek3v"  
https://app.formunauts.com/api/v2/fundraisers/ -d '{"first_name":  
"Lisi", "last_name": "Schreiber", "address": "Webgasse 4",  
"zip_code": "1030", "city": "Wien", "email": "test@test.com",  
"phone_number": "+43699123123213", "fundraiser_code": "0888",  
"customer": 6}'
```

Example response:

```
{  
  "id": 2440,  
  "first_name": "Lisi",  
  "last_name": "Schreiber",  
  "street": "",  
  "zip_code": "1030",  
  "city": "Wien",  
  "email": "test@test.com",  
  "phone_number": "+43699123123213",  
  "fundraiser_code": "0888",  
  "image_upload": null,  
  "createtime": "2016-05-04T07:26:14.512474Z",  
  "modifytime": "2016-05-04T07:26:14.512496Z",  
  "is_teamleader": false,  
  "is_coach": false,  
  "is_active": true,  
  "customer": 6,  
  "fundraiser_external_id": null,  
  "username": "demo-0888",  
  "language": "de",  
  "staffcloud_refid": null,  
  "staffcloud_import_date": null,  
  "image_thumbnail_url":  
  "https://assets.formunauts.com/static/img/default-avatar.png",  
}
```

Add a new fundraiser with password

URL: /fundraisers/

Method: POST

Data: Same as above, just add the Parameter password to the Request.

Example request:

```
curl -i -X POST -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" https://app.formunauts.com/api/v2/fundraisers/ -d '{"first_name": "Lisi", "last_name": "Schreiber", "street": "Webgasse 4", "zip_code": "1030", "city": "Wien", "email": "test@test.com", "phone_number": "+43699123123213", "fundraiser_code": "0888", "customer": 6, "password": "ijicIrryoad2"}'
```

Example response:

The response is the same as in the [Add a new fundraiser without fundraiser image](#) section.

Add a new fundraiser and upload a fundraiser image

Adds a new fundraiser and in the same request uploads an image of the fundraiser.

URL: /fundraisers/

Method: POST

Data:

The data is the same as in the [Add a new fundraiser without fundraiser image](#) section but as you want to also upload an image of the fundraiser it has to be with the "Content-Type: multipart/form-data" header set.

Example request:

```
curl -i -X POST -H "Content-Type: multipart/form-data" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" https://app.formunauts.com/api/v2/fundraisers/ -F
```

```
"image_upload=@/home/testuser/lisi_schreiber.jpg" -F
"first_name=Lisi" -F "last_name=Schreiber" -F "street=Webgasse 4" -F
"zip_code=1030" -F "city=Wien" -F "is_active=1" -F
"email=test@test.com" -F "phone_number="+43699123123213" -F
"customer=6" -F "fundraiser_code=1111"
```

Example response:

The response is the same as in the [Add a new fundraiser without fundraiser image](#) section.

Set/update the image of a fundraiser

Sets or updates the image of a fundraiser.

URL: /fundraisers/[fundraiser_id]/

Method: PATCH

Data:

Just set the field 'image_upload' to the new file. The data must be sent to the server as "Content-Type: multipart/form-data". Replace [fundraiser_id] in the URL with the actual fundraiser ID you got when creating the fundraiser.

Example request:

```
curl -i -X PATCH -H "Content-Type: multipart/form-data" -H
"Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v"
https://app.formunauts.com/api/v2/fundraisers/2440/ -F
"image_upload=@/home/testuser/new_fundraiser_image.jpg"
```

Example response:

```
{
  "id": 2440,
  "first_name": "Lisi",
  "last_name": "Schreiber",
  "street": "",
  "zip_code": "1030",
  "city": "Wien",
  "email": "test@test.com",
  "phone_number": "+43699123123213",
  "fundraiser_code": "0888",
```

```

      "image_upload":
"https://assets.formunauts.com/media/uploads/demo/fundraiser/0888_Lisi_SChraiber_04.02.2019_7208736c9a0b4da5ae62731afd96f24b.jpg",
      "createtime": "2016-05-04T07:26:14.512474Z",
      "modifytime": "2016-05-04T07:26:14.512496Z",
      "is_teamleader": false,
      "is_coach": false,
      "is_active": true,
      "customer": 6,
      "fundraiser_external_id": null,
      "username": "demo-0888",
      "language": "de",
      "staffcloud_refid": null,
      "staffcloud_import_date": null,
      "image_thumbnail_url":
"https://assets.formunauts.com/media/cache/25/68/25689ac1d26f19a2008e844bebdba313b.png",
    }

```

Set a fundraiser active/inactive

Sets a fundraiser into an inactive or active state. Inactive fundraisers can not login and can not submit donation forms.

URL: /fundraisers/[fundraiser_id]/

Method: PATCH

Data:

Just set the field `is_active` to 0 for inactive or 1 for active. Replace [fundraiser_id] in the URL with the actual fundraiser ID you got when creating the fundraiser.

Example request:

```

curl -i -X PATCH -H "Content-Type: multipart/form-data" -H
"Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v"
https://app.formunauts.com/api/v2/fundraisers/2440/ -F "is_active=0"

```

Example response:

```

{
  "id": 2440,
  "first_name": "Lisi",

```

```

    "last_name": "Schreiber",
    "street": "",
    "zip_code": "1030",
    "city": "Wien",
    "email": "test@test.com",
    "phone_number": "+43699123123213",
    "fundraiser_code": "0888",
    "image_upload": null,
    "createtime": "2016-05-04T07:26:14.512474Z",
    "modifytime": "2016-05-04T07:26:14.512496Z",
    "is_teamleader": false,
    "is_coach": false,
    "is_active": false,
    "customer": 6,
    "fundraiser_external_id": null,
    "username": "demo-0888",
    "language": "de",
    "staffcloud_refid": null,
    "staffcloud_import_date": null,
    "image_thumbnail_url":
"https://assets.formunauts.com/static/img/default-avatar.png",
}

```

Edit Fundraiser

You can use PATCH requests for partial updates and PUT requests for complete updates on existing fundraisers.

URL: /fundraisers/[fundraiser_id]/

Method: PUT, PATCH

Example request:

```

curl -i -X PATCH -H "Content-Type: application/json" -H "Accept:
application/json" -H "Authorization: Bearer
ZoGvK0RBdJnqxoItlqT9KnygmYek3v"
https://app.formunauts.com/api/v2/fundraisers/2440/ -d
'{"first_name": "Lizzi", "last_name": "Schraiber"}'

```

Example response:

```
{
  "id": 2440,
  "first_name": "Lizzi",
  "last_name": "Schraiber",
  "street": "",
  "zip_code": "1030",
  "city": "Wien",
  "email": "test@test.com",
  "phone_number": "+43699123123213",
  "fundraiser_code": "0888",
  "image_upload": null,
  "createtime": "2016-05-04T07:26:14.512474Z",
  "modifytime": "2018-09-20T16:23:14.762341Z",
  "is_teamleader": false,
  "is_coach": false,
  "is_active": false,
  "customer": 6,
  "fundraiser_external_id": null,
  "username": "demo-0888",
  "language": "de",
  "staffcloud_refid": null,
  "staffcloud_import_date": null,
  "image_thumbnail_url":
  "https://assets.formunauts.com/static/img/default-avatar.png",
}
```

List teams

Returns a list of teams. Add team id to request to display a single team.

URL: /teams/[team_id]

Method: GET

Example request:

```
curl -i -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" https://app.formunauts.com/api/v2/teams/
```

Example response:

```
{
  "count": 20,
  "total_pages": 2,
  "next": "https://app.formunauts.com/api/v2/teams/?page=2",
  "previous": null,
  "results": [{
    "id": 3240,
    "name": "Lisi Schreiber for Greenpeace UK",
    "start_date": "2016-06-01",
    "end_date": "2017-06-03",
    "tags": [],
    "team_type": "d2d",
    "campaign_type": "week_campaign",
    "prolong_workshift_setup": false,
    "goal_value_unit": "total_donations",
    "goal_value": 1000,
    "customer": 6,
    "campaign": 12,
    "petition_campaigns": [6],
    "image_thumbnail_url":
"https://assets.formunauts.com/media/cache/62/cc/62cca9ac.png",
    "leader": 2440,
    "coach": 2440,
    "assistant_coaches": [
      2441
```



```

],
"locations": [111, 112],
"team_fundraiser_work_shifts": [{
  "id": 24098,
  "date": "2016-06-01",
  "fundraiser": 2440,
  "work_shift": 76
}],
{
  "id": 24099,
  "date": "2016-06-01",
  "fundraiser": 2441,
  "work_shift": 76
}]
},
{
  "id": 1419,
  "name": "Miss Money Penny for Greenpeace DE",
  "start_date": "2015-06-14",
  "end_date": "2017-06-14",
  "tags": ["foo"],
  "team_type": null,
  "campaign_type": null,
  "prolong_workshift_setup": false,
  "goal_value_unit": "total_donations",
  "goal_value": 1000,
  "customer": 6,
  "campaign": 21,
  "petition_campaigns": [6],
  "image_thumbail_url":
"https://assets.formunauts.com/media/cache/62/cc/62cca9ac.png",
  "leader": 2112,
  "coaches": [
    2112
  ],
  "locations": [111, 112],
  "team_fundraiser_work_shifts": [{
    "id": 6099,
    "date": "2015-06-14",
    "fundraiser": 422,
    "work_shift": 17
  }],

```

```

{
    {
        "id": 6129,
        "date": "2015-06-14",
        "fundraiser": 428,
        "work_shift": 17
    }
}

```

Explanation of the JSON fields:

id	Integer	readonly	Internal unique ID
name	String	writable	Displayname for Team in Statistics and Admin Interface
start_date	String	writable required	First active date of Team in ISO format YYYY-MM-DD
end_date	String	writable required	Last active date of Team in ISO format YYYY-MM-DD
tags	List	writable	List of keywords associated with the team
team_type	String	writable	Either "d2d" (door2door), "infobooth", "private_site", "event" or NULL
campaign_type	String	writable	Either "city_campaign", "week_campaign" or NULL
prolong_workshift_setup	Bool	writable	Indicates if configured workshifts should be automatically renewed every day until the end_date of the team. Default: false
goal_value_unit	String	writable	"total_donations": sum of donation amounts "total_points": sum of donation points "supporter_count": count of donations
goal_value	Integer	writable	Positive Integer or null

customer	Integer	writable required	The internal unique ID of the customer this team is associated with
campaign	Integer	writeable	The internal unique ID of the campaign this team is associated with (see list of campaigns)
petition_campaigns	List	writeable	List of unique IDs of the petition_campaigns this team is associated with, empty array if none
image_thumbnail_url	String	readonly	Logo of the team, usually the logo of the campaign
leader	Integer	writeable	Internal unique ID of teamleader (see list of fundraisers)
coach	Integer	writeable	Internal unique ID of coach (see list of fundraisers)
assistant_coaches	List	writeable	Array of IDs of assistant coaches associated with this team (see list of fundraisers)
locations	List	writeable	Ids of assigned locations, query Listing available locations with the given ids for further informations
team_fundraiser_work_shifts	List	readonly	Array of Workshift Objects associated with this team: • "id": Integer, Internal Unique ID • "date": String, Date of Work • "Fundraiser": Integer, internal unique id of associated fundraiser • "Workshift": Integer, internal unique id of workshift description object (see list available workshifts)

NEW in version 1.19:

If location tracking is activated in the customers settings, the following data will be added to each team_fundraiser_work_shift record:

```
"locations":[{"createtime":"2018-06-22T09:22:56.086Z",  
"code":"hamburg-4","id":2,"name":"Hamburg 4"}]
```

createtime	String	Time when user has chosen given location
code	String	Location code
id	Integer	Internal id of location, for reference in /locations/ list
name	String	Name of location

Also two more fields are added to the `team_fundraiser_work_shift` record if location tracking is activated:

has_logged_in	Bool	Boolean, indicates if the fundraiser has actually logged in the according shift
donations_count	Integer	Count of donations of given fundraiser in the shift (new since Version 1.21)

NEW in version 2.0:

We added a `?date=YYYY-MM-DD` parameter where you can filter the list for teams active on that specific date (eg. `start_date` and `end_date` within the bounds of the given date), and `team_fundraiser_work_shifts` displayed only for that date. This way you can do incremental updates on your own database and don't need to query the whole list of teams.

NEW in version 2.2:

If the multi campaign setup is enabled on the customer the `campaign` (Integer) field is replaced by `campaigns` (List of Integers), so the campaign can be assigned to multiple campaigns, similar to how the `petition_campaigns` Field is working.

Add a new team

Adds a new team. If successful a HTTP status code 201 CREATED is returned.

URL: /teams/

Method: POST

Data:

This request expects you to post an JSON object. The post data must have the application/json content-type.

JSON Example:

```
{
  "start_date": "2016-10-10",
  "end_date": "2016-10-11",
  "customer": 6,
  "campaign": 21,
  "campaign_type": "city_campaign",
  "team_type": "d2d",
  "leader": 2440,
  "coach": 2440,
  "assistant_coaches": [2441],
  "locations": [111, 112],
  "tags": ["foo", "bar"],
  "name": "City D2D Team",
  "petition_campaigns": [],
  "prolong_workshift_setup": false,
  "goal_value_unit": "total_donations",
  "goal_value": 1000
}
```

Some remarks about the data:

- The name field was introduced in Version 1.13 of the API and is optional. If not set the Team name is constructed by the name of the teamleader and the campaign. If the name field is set the teamleader can be omitted (but not both of the fields at the same time!)
- The tags, petition_campaign, prolong_workshift_setup, goal_value_unit and goal_value fields were introduced in Version 1.14 and are all optional.

- The full List of possible writable values can be found in the [List teams](#) section

Example request:

```
curl -i -X POST -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" https://app.formunauts.com/api/v2/teams/ -d '{"start_date": "2016-10-10", "end_date": "2016-10-11", "customer": 6, "campaign": 21, "campaign_type": "city_campaign", "team_type": "d2d", "leader": 2440, "coaches": [2440], "tags": ["foo", "bar"], "team_fundraiser_work_shifts": [], "goal_value_unit": "total_donations", "goal_value": 1000}'
```

Example response:

```
{
  "id": 3538,
  "name": "City D2D Team",
  "start_date": "2016-10-10",
  "end_date": "2016-10-11",
  "team_type": "d2d",
  "campaign_type": "city_campaign",
  "customer": 6,
  "campaign": 21,
  "leader": 2440,
  "coach": 2440,
  "assistant_coaches": [2441],
  "tags": ["foo", "bar"],
  "team_fundraiser_work_shifts": [],
  "prolong_workshift_setup": false,
  "goal_value_unit": "total_donations",
  "goal_value": 1000,
  "image_thumbnail_url": null
}
```

Edit a team

Update an existing Team either via PUT (whole data is required) or PATCH (individual fields) Request.

Note: team_fundraiser_work_shifts is read only in the team view. Fundraiser Shifts can be edited under the /team/[team_id]/workshifts/ mount point (see below).

Note: after adding fundraiser_work_shifts to a Team, you can not change the campaign or petition_campaign fields anymore.

URL: /teams/[team_id]/

Methods: PUT, PATCH

Example request:

```
curl -i -X PATCH -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" https://app.formunauts.com/api/v2/teams/3538/ -d '{"start_date": "2016-10-10", "end_date": "2016-11-01"}'
```

Response:

Full updated Teamdata (same as in Add a new team)

Delete a team

Deletes a team. If the deletion is successful a HTTP code 204 NO CONTENT is returned.

Replace [team_id] in the URL with the actual team ID.

Important: Teams can only be deleted if they have not yet collected any donations or petitions.

If the team can not be deleted the API returns a HTTP code 403 FORBIDDEN.

URL: /teams/[team_id]/

Method: DELETE

Data:

Nothing

Example request:

```
curl -i -X DELETE -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" https://app.formunauts.com/api/v2/teams/3538/
```

Example response:

Empty response

Add a fundraiser shift

Add a fundraiser shift to an existing Team.

Note: Workshift date must be in bounds with the Team start_date and end_date. You should update the Team object (see Edit a Team) accordingly before adding a new shift.

URL: /teams/[team_id]/workshifts/

Methods: POST

Example request:

```
curl -i -X POST -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" https://app.formunauts.com/api/v2/teams/3538/workshifts -d '{"date": "2016-11-01", "fundraiser": 2444, "work_shift": 19}'
```

Example response:

```
{
  "id": 27309,
  "date": "2016-11-01",
  "fundraiser": 2444,
  "work_shift": 19
}
```


Edit a fundraiser shift

Edit an existing fundraiser work shift.

URL: /teams/[team_id]/workshifts/[workshift_id]/

Methods: PUT

Example request:

```
curl -i -X PUT -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" https://app.formunauts.com/api/v2/teams/3538/workshifts/27309/ -d '{"date": "2016-11-01", "fundraiser": 2444, "work_shift": 17}'
```

Example response:

```
{
  "id": 27309,
  "date": "2016-11-01",
  "fundraiser": 2444,
  "work_shift": 17
}
```

Delete a fundraiser shift

Delete an existing fundraiser work shift. If the deletion is successful a HTTP code 204 NO CONTENT is returned.

URL: /teams/[team_id]/workshifts/[workshift_id]/

Methods: DELETE

Example request:

```
curl -i -X DELETE -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer
```

ZoGvK0RBdJnqxoItlqT9KnygmYek3v"

<https://app.formunauts.com/api/v2/teams/3538/workshifts/27309/>

Example response:

Empty response

Listing available customers

Returns a list of customers. Add customer id to request to display a single customer.

URL: /customers/[customer_id]/

Method: GET

Example request:

```
curl -i -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" https://app.formunauts.com/api/v2/customers/
```

Example response:

```
{
  "count": 2,
  "next": null,
  "previous": null,
  "results": [{
    "currency": "€",
    "id": 7,
    "locale": "de_DE",
    "name": "Greenpeace",
    "points_label": "Punkte",
    "slug": "greenpeace"
  }, {
    "currency": "€",
    "id": 8,
    "locale": "de_DE",
    "name": "Ärzte ohne Grenzen",
    "points_label": "Punkte",
```

```
        "slug": "aerzte-ohne-grenzen"
    }
}
```

Explanation of the JSON fields:

id	Integer	Internal unique ID
currency	String	Currency symbol for transfered monetary values
locale	String	Language code of listed customer
name	String	Name of the customer
points_label	String	Points label for performance calculations
slug	String	Name of the customer that can be used in URLs

Listing available campaigns

Returns a list of campaigns. Add campaign id to request to display a single campaign.

URL: /campaigns/[campaign_id]/

Method: GET

Example request:

```
curl -i -H "Content-Type: application/json" -H "Accept:
application/json" -H "Authorization: Bearer
ZoGvK0RBdJnqxoItlqT9KnygmYek3v"
https://app.formunauts.com/api/v2/campaigns/
```

Example response:

```
{
  "count": 2,
  "next": null,
  "previous": null,
  "results": [{
    "id": 84,
    "name": " Greenpeace UK",
    "customer": 6,
    "is_online": true,
```

```

      "image_thumbnail_url":
"https://assets.formunauts.com/media/cache/c1/41/c14183d7bdf9d04f4eba
e2e87dc32161.png",
      "is_formunauts_one": true,
    }, {
      "id": 21,
      "name": "Greenpeace DE",
      "customer": 6,
      "is_online": true,
      "image_thumbnail_url":
"https://assets.formunauts.com/media/cache/62/cc/62cca9ae82f28127b207
fb4bec34843c.png",
      "is_formunauts_one": false,
    }
  ]
}

```

Explanation of the JSON fields:

id	Integer	Internal unique ID
name	String	Name of the campaign
customer	Integer	The ID of the customer this campaign is associated with
is_online	Bool	Flag indicating if the campaign is currently online
image_thumbnail_url	String	Generated thumbnail logo for the campaign
is_formunauts_one	Integer	Indicates if the campaign uses formunauts one features (<i>true</i>) or is a traditional campaign (<i>false</i>)

Listing available petition campaigns

Returns a list of petition campaigns. Add campaign id to request to display a single petition campaign.

URL: `/petition_campaigns/[campaign_id]/`

Method: GET

Example request:

```
curl -i -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" https://app.formunauts.com/api/v2/petition_campaigns/
```

Example response:

```
{
  "count": 2,
  "next": null,
  "previous": null,
  "results": [{
    "id": 84,
    "name": "Save the Whales!",
    "customer": 6,
    "is_online": true,
    "image_thumbnail_url":
    "https://assets.formunauts.com/media/cache/c2/2a/c22ab029b3dced16a8976121cacd70a9.png"
  }, {
    "id": 21,
    "name": "Save the Rainforest!",
    "customer": 6,
    "is_online": true,
    "image_thumbnail_url":
    "https://assets.formunauts.com/static/img/default-avatar.png"
  }]
}
```

Explanation of the JSON fields:

id	Integer	Internal unique ID
name	String	Name of the campaign
customer	Integer	The ID of the customer this campaign is associated with
is_online	Bool	Flag indicating if the campaign is currently online
image_thumbnail_url	String	Generated thumbnail logo for the campaign

Listing available work shifts

Returns a list of available work shifts.

URL: /workshifts/

Method: GET

Example request:

```
curl -i -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" https://app.formunauts.com/api/v2/workshifts/
```

Example response:

```
{
  "count": 5,
  "next": null,
  "previous": null,
  "results": [{
    "id": 17,
    "start_time": "07:00:00",
    "end_time": "19:00:00",
    "label": "Ganztag",
    "weight": 1.0,
    "customer": 6
  }, {
    "id": 18,
    "start_time": "07:00:00",
    "end_time": "12:00:00",
    "label": "Vormittag",
    "weight": 1.0,
    "customer": 6
  }, {
    "id": 19,
    "start_time": "13:00:00",
    "end_time": "19:00:00",
    "label": "Nachmittag",
    "weight": 1.0,
    "customer": 6
  }, {
    "id": 76,
    "start_time": "09:00:00",
    "end_time": "17:00:00",
```

```

        "label": "Full Day",
        "weight": 1.0,
        "customer": 54
    }, {
        "id": 77,
        "start_time": "09:00:00",
        "end_time": "13:00:00",
        "label": "Half Day",
        "weight": 0.6,
        "customer": 54
    }]
}

```

Explanation of the JSON fields:

id	Integer	Internal unique ID
start_time	String	Time when this work shift starts, ISO 8601 Format “HH:MM:SS”
end_time	String	Time when this work shift ends, ISO 8601 Format “HH:MM:SS”
label	String	Name of the work shift
weight	Float	Weight factor for calculating statistics
customer	Integer	The ID of the customer this work shift is associated with

Listing available locations

Returns a List of available Locations. Add location id to request to display a single location. The response differs depending on the customer setting ‘location module enabled’ (enables advanced location handling)

URL: /locations/[location_id]/

Method: GET

Example request:

```

curl -i -H "Content-Type: application/json" -H "Accept:
application/json" -H "Authorization: Bearer
ZoGvK0RBdJnqxoitlqT9KnygmYek3v"
https://app.formunauts.com/api/v2/locations/

```

Example response:

```
{
  "count": 2,
  "next": null,
  "previous": null,
  "results": [
    {
      "id": 112,
      "code": "A106",
      "name": "Mariahilferstrasse",
      "customer": 7,
      "is_active": true,
      "is_booth_required": false,
      "is_rain_proof": false,
      "type": "public",
      "capacity": 5,
      "price_per_day": 0,
      "street_address": "Mariahilerstrasse 37, 1060 Wien",
      "city": "Wien",
      "coordinates": [16.355874536143816, 48.200350485273944],
      "image":
"https://assets.formuntaus.com/uploads/demo/mariahilferstrasse.jpeg",
      "comments": [
        {
          "id": 123,
          "text": "Bester Standplatz der Stadt! Immer
hohe Frequenz",
          "user_id": 47,
          "full_name": "Alex Schwaiger",
          "createtime": "2020-07-16T08:16:43.490Z",
        }
      ],
      "latest_permit":
        {
          "id": 123,
          "file":
"https://assets.formunauts.com/private/demo/mariahilfer\_permit.pdf",
          "createtime": "2020-07-16T08:17:43.490Z",
        }
      ],
    }
  ]
}
```



```

    "id": 111,
    "code": "A105",
    "name": "Neubaugasse",
    "customer": 8,
  }
]
}

```

Explanation of the JSON fields:

id	Integer	readonly	Internal unique ID - needed for Reference in Team
code	String	writable required	Code of Location for external Reference
name	String	writable required	Name of Location for Display
customer	Integer	writable required	ID of associated Customer
is_active	Boolean	writable	Indicates if location is currently available - only available if customer has enabled location module
is_booth_required	Boolean	writable	Indicates if a booth is required on the given location (can be null) - only available if customer has enabled location module
is_rain_proof	Boolean	writable	Indicates if location can be used in bad weather conditions (can be null) - only available if customer has enabled location module
type	String	writable	either 'public', 'private' or 'unknown' - only available if customer has enabled location module - only available if customer has enabled location module
capacity	Integer	writable	Indicates how many fundraisers can work on the location at the same time (can be null) - only available if customer has enabled location module
price_per_day	Integer	writable	fee/rent per day for the location (can be null) - only available if customer has enabled location module

street_address	String	writable	Street Address of location (can be empty string) - only available if customer has enabled location module
city	String	writable	City the location is located in (can be empty string) - only available if customer has enabled location module
coordinates	List	writable	Array holding the longitude and latitude of the location (can be null) - only available if customer has enabled location module
image	String	writable	URL of image associated with location (can be null) - only available if customer has enabled location module
comments	Array	readonly	Array of comments, if any: • id (Integer) of the comment • text (String) of the comment • user_id (Integer) internal user id • full_name (String) full name of the user leaving the comment • Createtime (String - iso datetime) createtime of the comment - only available if customer has enabled location module
latest_permit	Object	readonly	Latest permits associated to the location, if any: • Id (Integer) • file (String): link to permit file • Createtime (String - iso datetime): upload time of permit file - only available if customer has enabled location module

Add a new location

Adds a new location. If successful a HTTP status code 201 CREATED is returned. The below example response is for a customer with 'location module' disabled. (see description in [Listing available Locations](#))

URL: /locations/

Method: POST

Data:

This request expects you to post an JSON object. The post data must have the application/json content-type.

Example request:

```
curl -i -X POST -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" https://app.formunauts.com/api/v2/locations/ -d '{"code": "A102", "name": "Wien Mitte", "customer": 7}'
```

Example response:

```
{
  "id": 108,
  "code": "A102",
  "name": "Wien Mitte",
  "customer": 7
}
```

Add a location with an image

Adds a new location and in the same request uploads an image of the location. This is only available if the 'location module' is activated on the according customer.

URL: /locations/

Method: POST

Data:

The data is the same as in the [Add a new location](#) section but as you want to also upload an image of the location it has to be with the Content-Type "multipart/form-data"

Example request:

```
curl -i -X POST -H "Content-Type: multipart/form-data" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" https://app.formunauts.com/api/v2/fundraisers/ -F "image=@/home/testuser/wien_mitte.jpg" -F "name=Wien Mitte" -F "code=A102" "customer=7"
```

Example response:

```
{
  "id": 108,
  "code": "A102",
  "name": "Wien Mitte",
  "customer": 7,
  "is_active": true,
  "is_booth_required": null,
  "is_rain_proof": null,
  "type": "unknown",
  "capacity": null,
  "price_per_day": null,
  "street_address": "",
  "city": "",
  "coordinates": null,
  "image":
"https://assets.formuntaus.com/uploads/demo/wien\_mitte.jpeg",
  "comments": [],
  "latest_permit": null,
}
```

Edit a location

Edit an existing Location. The below example response is for a customer with 'location module' disabled. (see description in [Listing available Locations](#))

URL: /locations/[location_id]/

Method: PATCH

Example request:

```
curl -i -X PATCH -H "Content-Type: application/json" -H "Accept:
application/json" -H "Authorization: Bearer
ZoGvK0RBdJnqxoItlqT9KnygmYek3v"
https://app.formunauts.com/api/v2/locations/10/ -d '{"code": "B102",
"name": "Wien Mitte Mall", "customer": 7}'
```

Example response:

```
{
  "id": 108,
  "code": "B102",
```

```
    "name": "Wien Mitte Mall",  
    "customer": 7  
}
```

Set/update the image of a location

Sets or updates the image of a location. This is only available if the 'location module' is activated on the according customer.

URL: /locations/[location_id]/

Method: PATCH

Data:

Just set the field 'image_upload' to the new file. The data must be sent to the server as Content-Type: multipart/form-data. Replace [location_id] in the URL with the actual location ID you got when creating the location.

Example request:

```
curl -i -X PATCH -H "Content-Type: multipart/form-data" -H
"Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v"
https://app.formunauts.com/api/v2/fundraisers/2440/ -F
"image=@/home/testuser/wien_mitte_2.jpg"
```

Example response:

```
{
  "id": 108,
  "code": "A102",
  "name": "Wien Mitte",
  "customer": 7,
  "is_active": true,
  "is_booth_required": null,
  "is_rain_proof": null,
  "type": "unknown",
  "capacity": null,
  "price_per_day": null,
  "street_address": "",
  "city": "",
  "coordinates": null,
  "image":
  "https://assets.formuntaus.com/uploads/demo/wien\_mitte\_2.jpeg",
  "comments": [],
  "latest_permit": null,
}
```

Delete a location

Deletes a location. If the deletion is successful a HTTP code 204 NO CONTENT is returned. Replace [location_id] in the URL with the actual location ID.

Important: Locations can only be deleted if they are not yet associated with any donations or petitions. If the team can not be deleted the API returns a HTTP code 403 FORBIDDEN.

URL: /locations/[location_id]/

Method: DELETE

Data:

Nothing

Example request:

```
curl -i -X DELETE -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" https://app.formunauts.com/api/v2/locations/108/
```

List job applications

Returns a list of the job applications in the system that are ready to be fetched via the API.

URL: /jobapplications/

Method: GET

Example request:

```
curl -i -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" https://app.formunauts.com/api/v2/jobapplications/
```

Example response:

```
{
  "count": 36,
  "total_pages": 4,
  "next":
"https://app.formunauts.com/api/v2/jobapplications/?page=2",
  "previous": null,
  "results": [{
    "uid": 1946646260,
    "campaign_id": 123,
    "fundraiser_code": "d-00789",
    "fundraiser_name": "Donut, James",
    "fundraiser_id": 47,
    "team_id": 1908,
    "customer_id": 9,
    "applicant_country": "at",
    "applicant_city": "Laa/Thaya",
    "applicant_workload": "5",
    "applicant_salutation": 1,
    "applicant_first_name": "Thomas",
    "applicant_last_name": "Mustermann",
    "applicant_email": "thomas.mustermann@example.com",
    "applicant_phone_number": "06762222222",
    "applicant_date_of_birth": "1990-01-01",
    "applicant_age_in_years": 28,
    "applicant_start_of_work": "2017-08-01",
    "createtime": "2017-07-20T09:43:56.458Z",
    "uploadtime": "2017-07-20T09:43:56.458Z",
    "comments": "",
    "consent_given": true,
    [additional fields]
  },
  { ... }
]
```

Explanation of the JSON fields:

uid	Integer	Unique ID of the donation
campaign_id	Integer	ID of the recruiting campaign this jobapplication is associated to. Can be used in Recruiting Campaign List to fetch further informations

fundraiser_code	String	Identification code of fundraiser
fundraiser_name	String	Name of fundraiser in format "last name, first name"
fundraiser_id	Integer	Reference to fundraiser can be used in Fundraiser List endpoint to get more data
team_id	Integer	id of the team as reference in the team lists endpoint
customer_id	Integer	id of the customer as reference in the customer lists endpoint
applicant_country	String	Two letter ISO country code like: AT, DE, UK
applicant_city	String	City name
applicant_workload	String	Entered workload in the form, depending on the jobtype configured in the campaign this is either in weeks (holiday job) or days (sidejob) NOTE: this field can be disabled in the form and will not be delivered in the API if it is disabled
applicant_salutation	Integer	1 -> "Herr" / "Mr" 2 -> "Frau" / "Mrs" 5 -> "Sonstiges" / "Other"
applicant_first_name	String	First Name of the applicant
applicant_last_name	String	Last Name of the applicant
applicant_email	String	Email Address of the applicant
applicant_phone_number	String	Phone Number of the application
applicant_date_of_birth	String	Birthday of the applicant ISO 8601 Format YYYY-MM-DD
applicant_age_in_years	Integer	Numeric value, age of applicant at signup.
applicant_start_of_work	String	Date when the applicant will be available for work. ISO 8601 Format YYYY-MM-DD NOTE: this field can be disabled in the form and will not be delivered in the API if it is disabled
createtime	String	Creation date and time of the application in ISO 8601 format.
uploadtime	String	Creation date and time of the application in ISO 8601

		format.
comments	String	Comments added on form
consent_given	Bool	If the customer is configured to include a consent checkbox on the recruiting form, this field will indicate if the consent was given to use the applicant's data (true) or not (false).

NEW in version 2.5:

In version 2.5 we added `?date_fro=yyyy-mm-dd` and `?date_to=yyyy-mm-dd` filters to the job applications endpoint. See Filtering section for more details.

Confirm retrieval of job applications

After you have fetched the job applications with the request above you have to confirm that you have received the job applications. This is done by sending an array of jobapplication uids to this endpoint. As a response you get an array of objects. Every object represents the status of the confirmation of one job applications.

URL: `/jobapplications/confirm_retrieval/`

Method: POST

Data:

This request expects you to post an array of JSON objects. The post data must have the `application/json` content-type.

The array must have the following form:

```
[
  {
    "uid": 1946646260,
    "status": "success",
    "message": ""
  },
  {
    "uid": 3554067970,
    "status": "error",
```

```

        "message": ""
    },
    { ... }
]

```

If you give a `status` of “success” the job application will be marked as retrieved and will not be included in the response of the `/jobapplications/` request anymore. The “message” field is ignored when status is “success”.

If you give a status of “error” you can give an error message to the server. This error message will be logged. The job application will continue to be listed in the donations returned by the `/jobapplications/` request.

Example request:

```

curl -i -X POST -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" -d '[{"uid": 1946646260,"status": "success","message": ""},{ "uid": 3554067970,"status": "error","message": ""},{ "uid": 123123123,"status": "success","message": ""}]'
https://app.formunauts.com/api/v2/jobapplications/confirm_retrieval/

```

Example response:

```

[
  {
    "status": "success",
    "message": "",
    "uid": 1946646260,
    "confirmation_date": "2017-07-20T15:10:54.921"
  },
  {
    "status": "error",
    "message": "",
    "uid": 3554067970
  },
  {
    "status": "error",
    "message": "The jobapplication with uid 123123123 does not exist.",
    "uid": 123123123
  }
]

```

```
}  
]
```

For every job application you get a status if the confirmation was successful. If the confirmation was successful you get the date and time of the confirmation. In case it was not successful you get an additional error message. Because of performance considerations we do not recommend confirming more than 100 job applications at once.

List Recruiting Campaigns

Returns a list of the activated recruiting campaigns in the system. Add recruitingcampaign id to the request to display a single recruitingcampaign

URL: /recruitingcampaigns/[recruitingcampaign_id]/

Method: GET

Example request:

```
curl -i -H "Content-Type: application/json" -H "Accept: application/json" -H "Authorization: Bearer ZoGvK0RBdJnqxoItlqT9KnygmYek3v" https://app.formunauts.com/api/v2/recruitingcampaigns/
```

Example response:

```
{
  "count": 8,
  "total_pages": 1,
  "next": null,
  "previous": null,
  "results": [{
    "id": 157,
    "name": "Städtekampagne Innsbruck",
    "min_age": 15,
    "applicant_min_workload": 2,
    "type": "City campaign",
    "jobtype": "sidejob",
    "description": "flexibel 2-6 Tage<br/>\r\nMontag - Samstag"
  }, {
    "id": 158,
    "name": "Reiseteam",
    "min_age": 15,
    "applicant_min_workload": 3,
    "type": "Travel team",
    "jobtype": "holiday job",
    "description": "Egal, wo du herkommst:<br/>\r\nSteig ein in unser Reiseteam!<br/>\r\nab 3 Wochen"
  },
  { ... }
]
```

}

Explanation of the JSON fields:

id	Integer	Unique ID of the recruiting campaign
name	String	Name of the recruiting campaign
min_age	Integer	Minimum Age of applicant (effects form validation)
applicant_min_workload	Integer	The minimum workload of the applicant. Depending on jobtype can either be in weeks (holiday job) or days per week (sidejob) NOTE: this field will not be delivered if the minimum workload field is configured not to appear on the form
type	String	Either 'Travel team' or 'City campaign'
jobtype	String	Either 'holiday job' or 'sidejob'
description	String	Free html description of the campaign (shown on form)

Webhooks

Webhooks are an easy way to notify another application of local changes in the data. As of Version 2.0 of the API we support both outgoing and incoming webhooks. Both are implemented as POST Requests in JSON Format.

Outgoing Webhooks

For receiving webhook events from app.formunauts.com you have to set up an URL on your system to receive webhook events.

To protect the URL from unauthorized access, we recommend using HTTP basic authentication.

After you have created an URL for receiving webhook events and you have protected it with basic authentication you need to send the `URL`, `username` and `password` to Formunauts.

Note that we expect the webhook endpoint at your side to return a HTTP status code in the 2xx number range, else a resending of the according webhook will be triggered.

Starting from the 2.0 Version of the API it will be possible to configure Webhooks in a custom JSON Format according to your wishes. The Webhooks can currently be configured to be sent either on a new `Donation` or `UnfinishedDonation`.

Get in contact with support@formunauts.com to set up a tailor made Webhook received by your API.

Incoming Webhooks

Starting from the 2.0 Version of the API we are also providing a webhook endpoint for incoming webhooks. The first intended use of the Webhook endpoint is to update retention data in our database, which we then can use to calculate statistics, bonus points, etc.

Differing to the other endpoints of the API we also offer Basic Authentication on the `/webhook/` endpoint to make the connection to the payment provider easier.

URL: `/webhook/`

Method: `POST`

Authentication: Basic Auth, Token Authentication

Example request:

```
curl -i -X POST -H "Content-Type: application/json" -H "Accept: application/json" -u "username:password" -d '{"event": "subscription_charged", "model": "donation", "uid": "1234567", "payload": {"date": "2021-04-27", "type": "payment", "status": "custom_status_code", "amount": 5000,}}' https://app.formunauts.com/api/v2/webhook/
```

JSON Example:

```
{
  "event": "subscription_charged",
  "model": "donation",
  "uid": 1234567,
  "payload": {
    "date": "2021-04-27",
    "type": "payment",
```

```

        "status": "custom_status",
        "amount": 5000,
    }
}

```

Explanation of the JSON fields:

event	String	required	currently available event types are • “subscription_charged” → send when the subscription was charged successfully • “subscription_charge_error” → send when the subscription was tried to be charged, but there was a payment error • “subscription_cancel” → send when the subscription was canceled by the donor • “refund” → send when a payment was refunded to the donor
model	String	optional	name of the model to update, as of the time being this is always “donation”
uid	Integer	required	This is the uid of the donation in our system, as retrieved in the List donations endpoint
date	String	required	date of the payment/cancellation in ISO format YYYY-MM-DD. this must be unique per donation and payment attempt.
type	String	required	either “payment” or “cancellation”
status	String	optional	custom status code provided by the caller to be saved with the payment, limited to 32 characters
amount	Integer	optional	Payment amount in cents, if available

Responses :

If the webhook is processed correctly a JSON object including a `success` message will be returned and the server will return a status code of 200. Else an according `error` message and a HTTP error code will be returned.

Appendix A: Status Codes

none	No status available
awaiting-validation	record is scheduled for validating
held	validation failed, record is held for manual editing
call	record needs confirmation through call center
approved	record was validated successfully or manually approved
conditionally-approved	record is marked for export though some problems exist with the data
failed	cancelled before it was exported to the charity
live	record was successfully exported to charity
cancelled	cancelled after it was exported to the charity
stopped	recurring donation stopped

Appendix B: Status Reasons

donation status reason should explain why a donation was set to its current status. It can be set in the following ways.

1. Through automated data validators
2. In the donation detail view for workflow enabled campaigns
3. When a donation is cancelled manually a reason has to be selected
4. When a call center import is processed

Each status reason should only be valid for one or two donation statuses. This is the current set of reasons grouped by their valid status.

reasons marked with * are also possible as values for `welcome_call_outcome`

Misc

- FCO-0-000, - *
- FCO-99-000, Multiple Status Reasons

Approved

- FCO-1-001, Long term confirmed *
- FCO-1-002, Converted One-off *
- FCO-1-003, Form sent *
- FCO-2-001, No time for call attempt *
- FCO-2-002, Wrong number *
- FCO-2-003, Not responded *
- FCO-2-004, Temporarily unavailable under this number *
- FCO-2-005, Donor voice mail requesting no call *
- FCO-2-006, Voice mail left by agent *
- FCO-2-007, Language problem *
- FCO-2-008, Refused *
- FCO-2-009, Unwell *
- FCO-2-010, Deaf *

Held

- FCO-2-011, Number of different person *
- FCO-3-001, Data change - lower donation amount *
- FCO-3-002, Data change - interval *
- FCO-3-999, Data change - other *
- FVO-10-001, Address Validation failed
- FVO-10-002, Quality Check failed
- FVO-10-003, Donor Feedback comment
- FVO-10-004, Status conflict
- FVO-10-005, Blacklisted
- FVO-10-006, Unknown Status Import

Cancelled or Failed

- FVO-1-001, Felt pressured *
- FVO-1-002, Cant Afford *
- FVO-1-003, Spouse Objected *
- FVO-1-004, Changed Mind *
- FVO-1-005, Doesn't remember signing up *
- FVO-1-006, Told one-off *

- FVO-1-007, Not long term *
- FVO-1-008, Cancel without reason *
- FVO-1-009, Told just petition (leads) *
- FVO-1-010, Gone Away *
- FVO-1-011, Considered Vulnerable *
- FVO-1-012, Downgrade under minimum *
- FVO-1-013, Deceased *
- FVO-1-014, Mystery shopper *
- FVO-1-015, Test Donation *
- FVO-1-016, Made redundant *
- FVO-1-017, Unemployed *
- FVO-2-001, Potential fraud
- FVO-2-002, Duplicate Donor
- FVO-2-003, Friends and Family
- FVO-2-004, Same account / different details
- FVO-2-005, Account name differs
- FVO-3-001, Number missing (lead)
- FVO-3-002, E-Mail missing
- FVO-3-003, Opt-in missing
- FVO-3-004, Self checkout unsubscribed (leads)
- FVO-4-001, Max amount attempts
- FVO-4-002, Invalid Bank Details
- FVO-4-003, Account Closed
- FVO-4-004, Changing Banks
- FVO-4-005, Direct Debits not permissible on this Account
- FVO-4-006, DD refund claimed
- FVO-4-007, No Instruction
- FVO-4-011, Transaction code/user status incompatible
- FVO-4-008, Instruction Cancelled
- FVO-4-009, Instruction expired
- FVO-4-010, Payer Reference is not unique